ΕΡΓΑΣΙΑ 1η

ΠΡΟΣΤΑΣΊΑ ΑΡΧΕΊΩΝ (*FILE PROTECTION*)

ΔΡΟΣΟΣ ΚΑΤΣΙΜΠΡΑΣ

Μάθημα: Κυβερνοασφάλεια
ΠΜΣ «Προηγμένες Τεχνολογίες Πληροφορικής και Εφαρμογές»

Ακαδημαϊκό μέλος μαθήματος:
Καθηγητής κ. Ριζομυλιώτης Παναγιώτης

ΧΑΡΟΚΟΠΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΜΑΤΙΚΗΣ

Τμήμα Πληροφορικής και Τηλεματικής
Χαροκόπειο Πανεπιστήμιο Αθηνών

Απρίλιος, 2026

Προστασία αρχείων (*File Protection*)

ΔΡΟΣΟΣ ΚΑΤΣΙΜΠΡΑΣ

Περίληψη

Η παρούσα εργασία πραγματεύεται τον σχεδιασμό και την υλοποίηση ενός ολοκληρωμένου συστήματος προστασίας ψηφιακών αρχείων, εστιάζοντας στη διασφάλιση της εμπιστευτικότητας και της ακεραιότητας των δεδομένων. Η προτεινόμενη λύση υιοθετεί μια δυναμική προσέγγιση διαχείρισης κλειδιών μέσω μηχανισμού ανακύκλωσης κλειδιών (Key Rotation), όπου κάθε αρχείο προστατεύεται από ένα μοναδικό κρυπτογραφικό κλειδί που ανανεώνεται μετά από κάθε διαδικασία ελέγχου ή αποκρυπτογράφησης. Για την παραγωγή των κλειδιών χρησιμοποιείται μια ιεραρχική δομή βασισμένη σε συναρτήσεις παραγωγής κλειδιών (Key Derivation Functions - KDFs) από ένα κεντρικό κλειδί (Master Key). Συνεπώς, η υλοποίηση επιτρέπει στον χρήστη την παραμετροποίηση κρίσιμων παραγόντων, όπως η επιλογή κρυπτογραφικών αλγορίθμων (AES-GCM, ChaCha20-Poly1305), το επίπεδο ασφαλείας (128/256 bits) και ο στόχος ασφάλειας. Τέλος, η εργασία αναλύει τις κρυπτογραφικές επιλογές με γνώμονα την ανθεκτικότητα σε σύγχρονες απειλές και την ευελιξία (Cryptographic Agility), προσφέροντας ένα στιβαρό πλαίσιο για την προστασία δεδομένων.

Λέξεις Κλειδιά: Cryptography, Data Integrity, Confidentiality, Key Rotation, Key Derivation Functions (KDF), Authenticated Encryption (AEAD), File Protection, Security Engineering.

Περιεχόμενα

Περίληψη ii

Κατάλογος Σχημάτων vii

Κατάλαγος Πινάκων ix

Λίστα Συντομογραφιών xi

1 Κεφάλαιο 1: Εισαγωγή και Σκοπιμότητα 1
- 1.1 Αντικείμενο της Εργασίας . . . 1
- 1.2 Σκοπιμότητα και Σενάριο Χρήσης (Business Scenario) . . . 1
- 1.3 Στόχοι του Project . . . 2
- 1.4 Μεθοδολογία υλοποίησης . . . 2

2 Κεφάλαιο 2: Θεωρητικό Υπόβαθρο και Κρυπτογραφική Ανάλυση 4
- 2.1 Στόχοι Προστασίας και η Παράμετρος Ασφαλείας (λ) . . . 4
- 2.2 Κωδικοί Επαλήθευσης (MACs) και Authenticated Encryption (AEAD) . . 4
  - 2.2.1 AES-GCM: Block Ciphers, Counter Mode και Hardware Acceleration 5
  - 2.2.2 ChaCha20-Poly1305: Stream Ciphers και η αρχιτεκτονική ARX . . 6
  - 2.2.3 Σύγκριση και Κρυπτογραφική Ευελιξία (Crypto-Agility) . . . 7
- 2.3 Συναρτήσεις Παραγωγής Κλειδιών (KDF) και Διαχείριση Εντροπίας . . . 8
- 2.4 Η Στρατηγική του Key Rotation και η αποφυγή Key/Nonce Reuse . . . 9

3 Κεφάλαιο 3: Σχεδιασμός και Αρχιτεκτονική Συστήματος 11
- 3.1 Αρχιτεκτονική Ιεραρχίας Κλειδιών (Key Hierarchy) . . . 11
  - 3.1.1 Root Key (Master Password) και Entropy . . . 12
  - 3.1.2 Derived File Keys: Η απομόνωση των κλειδιών . . . 12
- 3.2 Σχεδιασμός του Μηχανισμού Key Rotation *(State Management)* . . . 12
  - 3.2.1 Trigger Events: Rotation κατά το Decryption vs Integrity Check . 12
  - 3.2.2 Forward Secrecy: Προστατεύοντας το παρελθόν . . . 13
- 3.3 Διάγραμμα Ροής Λειτουργίας (Workflow Diagram) . . . 13
- 3.4 Διαχείριση Μεταδεδομένων (Metadata Headers) . . . 15

4 Κεφάλαιο 4: Υλοποίηση και Ανάλυση Κώδικα 16
- 4.1 Περιβάλλον Ανάπτυξης και Βιβλιοθήκες (Python Cryptography) . . . 16
- 4.2 Ανάλυση της Κλάσης Προστασίας Αρχείων . . . 17
  - 4.2.1 Παραγωγή Κρυπτογραφικού Υλικού (`_derive_master_key`) . . . 17
  - 4.2.2 Διαδικασία Κρυπτογράφησης και Δομή Metadata . . . 17

4.2.3 Μηχανισμός Ανανέωσης Κλειδιών (Key Rotation) 18
4.3 Υλοποίηση Δυναμικής Παραμετροποίησης (Crypto-Agility) 18
4.3.1 Επιλογή Αλγορίθμου: Υλοποίηση AES-GCM vs ChaCha20-Poly1305 18
4.3.2 Επιλογή Παραμέτρου Ασφαλείας: Στόχοι $\lambda = 128$ και $\lambda = 256$ bits 19
4.4 Χειρισμός Εξαιρέσεων και Ασφάλεια Λαθών 19
4.4.1 Προστασία Ακεραιότητας και Αυθεντικοποίησης 19
4.4.2 Διαχείριση Σφαλμάτων κατά την Αποκρυπτογράφηση και το Key Rotation 20

5 Κεφάλαιο 5: Αξιολόγηση και Δοκιμές (Testing) 22
5.1 Σενάρια Ορθής Λειτουργίας (Positive Testing) 22
5.2 Έλεγχος Παραβίασης Ακεραιότητας (Tamper Detection) 23
5.3 Δοκιμές Key Rotation (Verification of Key Change) 23
5.4 Δυνατότητα Συνεχόμενης Επεξεργασίας (CLI Loop) 24

6 Κεφάλαιο 6: Τεχνικές Προκλήσεις 25
6.1 Αντιμετώπιση Προβλημάτων (Troubleshooting) 25
6.2 Σύγκριση Αποδοτικότητας Αλγορίθμων 25
6.3 Περιορισμοί του Συστήματος 27

Συμπεράσματα και Μελλοντικές Επεκτάσεις 27

Παράρτημα A : Python Code 29

Παράρτημα B : Αποθετήριο Κώδικα (GitHub) 32

Βιβλιογραφικές Αναφορές 34

Κατάλογος Σχημάτων

1.1 Ροή εργασίας Μεθοδολογίας 3
2.1 Θεωρητικό μοντέλο λειτουργίας Authenticated Encryption with Associated Data (AEAD) 5
2.2 Θεωρητικό μοντέλο λειτουργίας Συνάρτησης Παραγωγής Κλειδιών (KDF) με χρήση Salt και Επαναλήψεων. 9
3.1 Δενδρική αναπαράσταση της ιεραρχίας κλειδιών. 11
3.2 Διάγραμμα κατάστασης ανακύκλωσης κλειδιών (Key Rotation State) και συνθήκες πυροδότησης. 13
3.3 Αρχιτεκτονική του συστήματος προστασίας αρχείων και διάγραμμα ροής δεδομένων (Data Flow Diagram). 14
4.1 Σχηματική αναπαράσταση της σειριακής δομής των Metadata και του Ciphertext στο κρυπτογραφημένο αρχείο. 18
4.2 Ασφαλής διαχείριση του `InvalidSignature` exception σε HMAC-SHA256 Mode. 19
4.3 Αποτυχία αυθεντικοποίησης (InvalidTag) σε AEAD σχήμα λόγω λανθασμένου password. 20
4.4 Εντοπισμός και διαχείριση σφάλματος μεγέθους Nonce κατά τη διαδικασία αποκρυπτογράφησης σε AES-CTR Mode. 20
4.5 Αποτυχία της διαδικασίας Key Rotation λόγω λανθασμένου παλαιού κωδικού. 21
5.1 Επιτυχής κρυπτογράφηση αρχείου και διαγραφή του αρχικού για λόγους ασφαλείας. 22
5.2 Επιτυχής επαλήθευση ακεραιότητας, ανάκτηση δεδομένων και αυτόματη εναλλαγή κλειδιού (Key Rotation). 23
5.3 Απόρριψη αποκρυπτογράφησης μετά από ανίχνευση αλλοίωσης στα δεδομένα του αρχείου (Tamper Detection). 23
5.4 Επιτυχής ανανέωση κρυπτογραφικού κλειδιού με χρήση του αλγορίθμου ChaCha20-Poly1305 και παραγωγή νέου Master Key. 24
5.5 Εμφάνιση του ανανεωμένου μενού και δυνατότητα επιλογής επόμενου αρχείου ή τερματισμού. 24
6.1 Συγκριτική ανάλυση κρυπτογραφικών προτύπων λειτουργίας (Modes of Operation) ως προς την ασφάλεια και την αποδοτικότητα. 26

Κατάλογος Πινάκων

1 Σύγκριση Αλγορίθμων Authenticated Encryption (AES-GCM vs ChaCha20) 8
2 Ανάλυση Κρυπτογραφικών Μηχανισμών και Στόχων Προστασίας 29
3 Προδιαγραφές Συναρτήσεων Κρυπτογραφικού Μηχανισμού 30

Λίστα Συντομογραφιών

AEAD	Authenticated Encryption with Associated Data
AES	Advanced Encryption Standard
API	Application Programming Interface
CLI	Command Line Interface
CSPRNG	Cryptographically Secure Pseudo-Random Number Generator
CTR	Counter Mode
GCM	Galois/Counter Mode
HMAC	Hash-based Message Authentication Code
IV	Initialization Vector
JSON	JavaScript Object Notation
KDF	Key Derivation Function
MK	Master Key
Nonce	Number used Once
PBKDF2	Password-Based Key Derivation Function 2
SHA	Secure Hash Algorithm

1 Κεφάλαιο 1: Εισαγωγή και Σκοπιμότητα

Στο παρόν αρχικό κεφάλαιο παρατίθεται το πλαίσιο της εργασίας, προσδιορίζοντας το αντικείμενο μελέτης και τη σκοπιμότητα ανάπτυξης ενός συστήματος δυναμικής προστασίας αρχείων. Αρχικά, οριοθετούνται οι βασικές αρχές ασφαλείας που καλείται να υπηρετήσει η υλοποίηση, ενώ στη συνέχεια παρουσιάζεται το επιχειρησιακό σενάριο χρήσης που αναδεικνύει την ανάγκη για προηγμένη διαχείριση κλειδιών. Τέλος, αναλύονται οι συγκεκριμένοι στόχοι του έργου και η μεθοδολογία που ακολουθήθηκε, προσφέροντας μια συνολική εικόνα της προσέγγισης που υιοθετήθηκε για την αντιμετώπιση των προκλήσεων που θέτει η σύγχρονη κρυπτογραφία σε περιβάλλοντα αποθήκευσης δεδομένων.

1.1 Αντικείμενο της Εργασίας

Το αντικείμενο της παρούσας εργασίας επικεντρώνεται στον σχεδιασμό, την αρχιτεκτονική ανάλυση και την προγραμματιστική υλοποίηση ενός συστήματος ασφαλείας για την προστασία δεδομένων. Η ανάγκη για την προστασία ψηφιακών αρχείων αποτελεί κρίσιμη συνιστώσα στη σύγχρονη κυβερνοασφάλεια, καθώς οι παραδοσιακές μέθοδοι στατικής κρυπτογράφησης συχνά αποδεικνύονται ελλιπείς έναντι προηγμένων απειλών.

1.2 Σκοπιμότητα και Σενάριο Χρήσης (Business Scenario)

Η ανάγκη για την ανάπτυξη της παρούσας λύσης απορρέει από τις προκλήσεις που αντιμετωπίζουν οι οργανισμοί στη διαχείριση ευαίσθητων ψηφιακών περιουσιακών στοιχείων. Σε ένα τυπικό επιχειρησιακό σενάριο, μια εταιρεία αποθηκεύει κρίσιμα αρχεία παραμέτρων (configuration files) ή βάσεις δεδομένων που περιέχουν προσωπικά δεδομένα. Η παραδοσιακή προσέγγιση της χρήσης ενός στατικού κλειδιού κρυπτογράφησης ενέχει τον κίνδυνο της «κρυπτογραφικής εξάντλησης» και της πλήρους έκθεσης των δεδομένων σε περίπτωση που το κλειδί διαρρεύσει ή υποκλαπεί.

Η σκοπιμότητα του προτεινόμενου συστήματος εστιάζεται στους εξής άξονες:

- Περιορισμός της Έκθεσης *(Mitigating Key Compromise)*: Μέσω του μηχανισμού ***Key Rotation***, ακόμα και αν ένας επιτιθέμενος καταφέρει να υποκλέψει το κλειδί μιας συγκεκριμένης χρονικής στιγμής, η πρόσβασή του περιορίζεται αποκλειστικά σε εκείνη την έκδοση του αρχείου. Η επόμενη ανάγνωση ή τροποποίηση από τον νόμιμο χρήστη θα καταστήσει το προηγούμενο κλειδί, άχρηστο.
- Κανονιστική Συμμόρφωση *(Compliance)*: Σύγχρονα πρότυπα όπως το GDPR και το PCI DSS απαιτούν αυστηρούς μηχανισμούς ελέγχου πρόσβασης και τακτική ανανέωση των κρυπτογραφικών κλειδιών. Η αυτοματοποιημένη ανακύκλωση κλειδιών αποτελεί βέλτιστη πρακτική (best practice) για την τήρηση αυτών των προτύπων.
- Προστασία από Εσωτερικές Απειλές *(Insider Threats)*: Η χρήση ενός Master Key (ή passphrase) που παράγει δυναμικά κλειδιά ανά αρχείο, διασφαλίζει ότι η πρόσβαση στα δεδομένα απαιτεί πάντα την κατοχή του κεντρικού μυστικού, αποτρέποντας την απλή αντιγραφή κρυπτογραφημένων αρχείων για μελλοντική επίθεση offline.

Συνεπώς, η υλοποίηση δεν αποτελεί απλώς μια προγραμματιστική άσκηση, αλλά μια προσέγγιση Defense in Depth (Άμυνα σε Βάθος), που στοχεύει στη διατήρηση της εμπιστευτικότητας και της ακεραιότητας των αρχείων καθ' όλο τον κύκλο ζωής τους.

1.3 Στόχοι του Project

Οι στόχοι της παρούσας εργασίας ιεραρχούνται με βάση τις αρχές της Κρυπτογραφικής Μηχανικής (Cryptographic Engineering) και τις απαιτήσεις της εκφώνησης. Το σύστημα στοχεύει στην παροχή μιας πολυεπίπεδης λύσης, η οποία δεν περιορίζεται στην απλή εκτέλεση συναρτήσεων, αλλά στην ορθή διαχείριση του κρυπτογραφικού κινδύνου.

Οι κύριοι στόχοι συνοψίζονται στα κάτωθι:

- **Διασφάλιση Ακεραιότητας και Εμπιστευτικότητας**: Η υλοποίηση μηχανισμών που εγγυώνται ότι τα αρχεία παραμένουν αδιάβλητα από μη εξουσιοδοτημένες αλλαγές (Integrity) και μη προσβάσιμα σε τρίτους (Confidentiality).
- **Υλοποίηση Δυναμικού Key Lifecycle (Key Rotation)**: Η ανάπτυξη μιας ροής εργασίας όπου το κρυπτογραφικό κλειδί κάθε αρχείου έχει περιορισμένο χρόνο ζωής, επιβάλλοντας την αναγέννησή του μετά από κάθε λειτουργία (Encryption/Decryption/Verification).
- **Παραμετροποιήσιμη Ασφάλεια (Security Agility)**: Η παροχή δυνατότητας στον χρήστη να επιλέγει το επίπεδο ασφάλειας ($\lambda \in \{128, 256\}$ bits) και τους επιθυμητούς αλγορίθμους, επιτρέποντας στο σύστημα να προσαρμόζεται σε διαφορετικά επίπεδα απειλών.
- **Ασφαλής Παραγωγή Κλειδιών (Key Derivation)**: Να υπάρχει ορθή χρήση δευτερευόντων κλειδιών που παράγονται από ένα κεντρικό Master Key ή Passphrase, αποτρέποντας την απευθείας έκθεση του κύριου μυστικού σε κρυπτογραφικές πράξεις.
- **Αρχιτεκτονική Modular Σχεδιασμού**: Κλείνοντας, είναι σημαντικό να αναφερθεί πως η δόμηση του κώδικα χρειάζεται να γίνει με τέτοιο τρόπο ώστε να είναι εύκολη η αντικατάσταση ή η αναβάθμιση των κρυπτογραφικών βιβλιοθηκών (π.χ. μετάβαση από cryptography σε PyNaCl), ακολουθώντας τα πρότυπα του software engineering.

Ένας παράλληλος, και εξίσου σημαντικός, στόχος της εργασίας είναι η εκπαιδευτική εμβάθυνση στις ιδιότητες των αλγορίθμων (όπως η ανθεκτικότητα σε collision attacks ή η σημασία του initialization vector), διασφαλίζοντας ότι η τελική υλοποίηση αποτελεί προϊόν βαθιάς κατανόησης του θεωρητικού υπόβαθρου της κυβερνοασφάλειας.

1.4 Μεθοδολογία υλοποίησης

Σε αυτή την υποενότητα, σημαντικό είναι να αναφερθούμε στην μεθοδολογία που θα χρησιμοποιηθεί για την υλοποίηση της. Θα ακολουθήσουμε μια σταδιακή προσέγγιση **(Phased Implementation)**. Για την επιτυχή ολοκλήρωση της εργασίας και τη διασφάλιση της ορθότητας των κρυπτογραφικών λειτουργιών, υιοθετήθηκε μια μεθοδολογία, η οποία διασφαλίζει τη μετάβαση από το θεωρητικό επίπεδο στην πρακτική εφαρμογή με ασφάλεια και συνέπεια.

- ***Στάδιο 1: Θεωρητική Ανάλυση και Επιλογή Πρωτοκόλλων***
  Στο πρώτο στάδιο πραγματοποιήθηκε μελέτη των κρυπτογραφικών προτύπων (NIST, IETF) για την επιλογή των καταλληλότερων αλγορίθμων AEAD (Authenticated Encryption with Associated Data). Παράλληλα, αναλύθηκαν οι διαφορές μεταξύ AES-GCM και ChaCha20-Poly1305, καθώς και η μαθηματική βάση των KDF (PBKDF2 /Argon2) για την ασφαλή διαχείριση του Master Key.

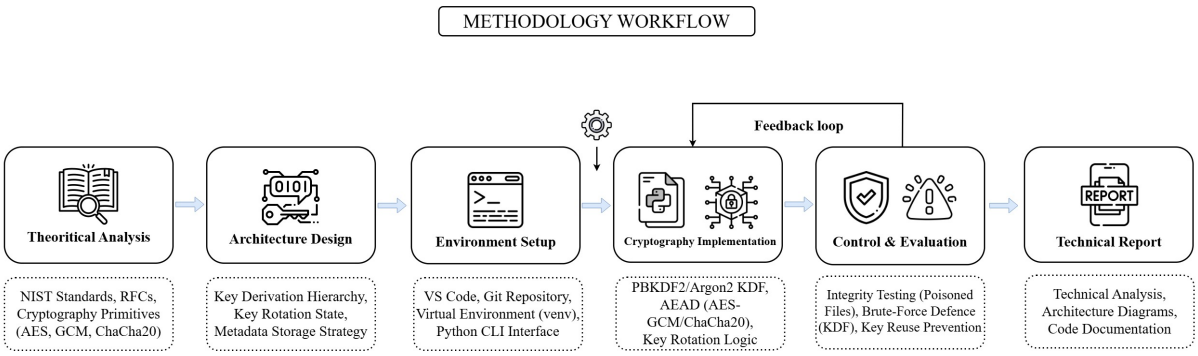
- ***Στάδιο 2: Σχεδιασμός Αρχιτεκτονικής και Ροής Δεδομένων***
  Σε αυτό το σημείο, σχεδιάστηκε το λογικό διάγραμμα της εφαρμογής, δίνοντας έμφαση στον μηχανισμό Key Rotation. Έτσι, καθορίστηκε ο τρόπος αποθήκευσης των μεταδεδομένων (salts, nonces) και η ιεραρχική δομή παραγωγής κλειδιών, ώστε να ικανοποιείται η απαίτηση για μοναδικό κλειδί ανά αρχείο και ανά χρήση.

- ***Στάδιο 3: Προετοιμασία Περιβάλλοντος Ανάπτυξης και Διεπαφής (Setup & CLI)*** Εγκαταστάθηκε και παραμετροποιήθηκε το περιβάλλον ανάπτυξης (Virtual Environment, IDE) και δημιουργήθηκε το σύστημα ελέγχου εκδόσεων (Git Repository) για την ασφαλή διαχείριση του κώδικα. Παράλληλα, σχεδιάστηκε η διεπαφή Γραμμής Εντολών (CLI) μέσω της οποίας ο χρήστης αλληλεπιδρά ομαλά με τις λειτουργίες του συστήματος.

- ***Στάδιο 4: Προγραμματιστική Υλοποίηση (Development)***
  Η υλοποίηση πραγματοποιήθηκε σε περιβάλλον Python, χρησιμοποιώντας τη βιβλιοθήκη ***cryptography***. Έπειτα, ενσωματώθηκαν οι αρχές του αμυντικού προγραμματισμού (defensive programming), με αυστηρό έλεγχο σφαλμάτων (exception handling) για την αποτροπή κρυπτογραφικών διαρροών (π.χ. padding oracle attacks).

- ***Στάδιο 5: Έλεγχος Ακεραιότητας και Δοκιμές (Testing/Validation)***
  Το σύστημα υποβλήθηκε σε σενάρια ελέγχου, όπως η προσπάθεια αποκρυπτογράφησης με λανθασμένο Master Key και η χειροκίνητη αλλοίωση bit στο κρυπτογραφημένο αρχείο, για την επιβεβαίωση της άμεσης ανίχνευσης παραβίασης της ακεραιότητας.

- ***Στάδιο 6: Τεκμηρίωση και Ακαδημαϊκή Συγγραφή***
  Στο τελικό στάδιο, δημιουργήθηκε το report της εργασίας, με στόχο την πλήρη αποτύπωση των σχεδιαστικών αποφάσεων και την ανάλυση των κρυπτογραφικών επιλογών που καθιστούν το σύστημα στιβαρό και επεκτάσιμο.



Εικόνα 1.1: Ροή εργασίας Μεθοδολογίας

2 Κεφάλαιο 2: Θεωρητικό Υπόβαθρο και Κρυπτογραφική Ανάλυση

Στο παρόν κεφάλαιο αναλύεται το θεωρητικό υπόβαθρο που διέπει τις σχεδιαστικές επιλογές του συστήματος, σε άμεση συνάρτηση με τους στόχους της εργασίας. Βασιζόμενοι στις σύγχρονες κρυπτογραφικές αρχές, εξετάζονται οι μηχανισμοί που διασφαλίζουν την ακεραιότητα και την εμπιστευτικότητα των δεδομένων. Ιδιαίτερη έμφαση δίνεται στην παράμετρο ασφαλείας λ, στην κατηγορία αλγορίθμων Authenticated Encryption (AEAD), καθώς και στις συναρτήσεις παραγωγής κλειδιών (KDFs) που αποτελούν τον πυρήνα του μηχανισμού Key Rotation. Συνεπώς, στόχος μας είναι η πλήρης τεκμηρίωση των κρυπτογραφικών πρωτοκόλλων που υλοποιήθηκαν, έναντι γνωστών μοντέλων απειλών.

2.1 Στόχοι Προστασίας και η Παράμετρος Ασφαλείας (λ)

Σύμφωνα με το θεωρητικό πλαίσιο της σύγχρονης κρυπτογραφίας, ο σχεδιασμός κάθε ασφαλούς συστήματος καθορίζεται από τους θεμελιώδεις στόχους Προστασίας (Protection Goals). Η παρούσα υλοποίηση προσαρμόζει αυτούς τους στόχους στις απαιτήσεις της προστασίας δεδομένων σε κατάσταση ηρεμίας (Data-at-Rest), εστιάζοντας στους εξής δύο άξονες:

- **Εμπιστευτικότητα *(Confidentiality)***: Αφορά τη διασφάλιση ότι τα δεδομένα του αρχείου παραμένουν ακατάληπτα σε περίπτωση μη εξουσιοδοτημένης πρόσβασης. Η ισχύς της εμπιστευτικότητας ποσοτικοποιείται μαθηματικά μέσω της παραμέτρου ασφαλείας λ. Το προτεινόμενο σύστημα υποστηρίζει δυναμική παραμετροποίηση με $\lambda = 128$ bits (στάνταρ βιομηχανική πρακτική, βέλτιστη απόδοση) και $\lambda = 256$ bits (υψηλότερο επίπεδο ασφάλειας, ανθεκτικότητα σε μελλοντικές επιθέσεις και κβαντικούς υπολογιστές).
- **Ακεραιότητα *(Integrity)***: Εξασφαλίζει ότι το περιεχόμενο ενός αρχείου παραμένει απόλυτα αναλλοίωτο από τη στιγμή της αποθήκευσής του. Το σύστημα είναι σχεδιασμένο να ανιχνεύει οποιαδήποτε, έστω και ελάχιστη, τροποποίηση σε επίπεδο bit (bit-flipping) στο κρυπτογραφημένο κείμενο και να απορρίπτει το αρχείο, κάνοντας χρήση Κωδικών Πιστοποίησης (Message Authentication Codes - MACs).

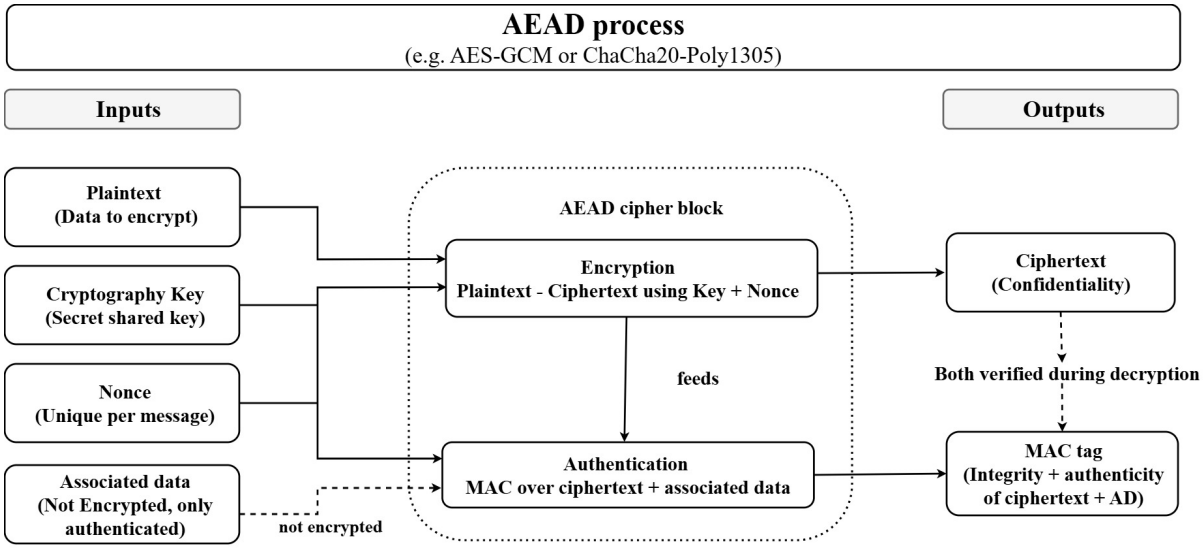
Σύμφωνα και με τα απαιτούμενα της εργασίας, η αρχιτεκτονική του συστήματος επιτρέπει στον χρήστη να ορίσει τον ακριβή στόχο ασφαλείας ανάλογα με τη φύση των δεδομένων: είτε την προστασία αποκλειστικά της ακεραιότητας (π.χ. για αρχεία καταγραφής - log files), είτε τον συνδυασμό αποκρυπτογράφησης και επαλήθευσης ακεραιότητας για την πλήρη προστασία ευαίσθητων αρχείων.

2.2 Κωδικοί Επαλήθευσης (MACs) και Authenticated Encryption (AEAD)

Η παραδοσιακή προσέγγιση στην κρυπτογραφία αντιμετώπιζε την εμπιστευτικότητα και την ακεραιότητα ως δύο ξεχωριστά και ανεξάρτητα προβλήματα. Όπως αναλύθηκε στο θεωρητικό πλαίσιο του μαθήματος (σχετικά με τους Κώδικες Πιστοποίησης Μηνύματος -

MACs), ο συνδυασμός ενός αλγορίθμου κρυπτογράφησης με έναν ξεχωριστό αλγόριθμο MAC (όπως η προσέγγιση Encrypt-then-MAC) απαιτεί εξαιρετικά προσεκτική υλοποίηση. Λάθη σε αυτή τη σύνθεση οδήγησαν ιστορικά σε σοβαρές ευπάθειες, όπως τα padding oracle attacks.

Για την επίλυση αυτών των προβλημάτων, η σύγχρονη κρυπτογραφική μηχανική επιτάσσει τη χρήση σχημάτων ***Authenticated Encryption with Associated Data (AEAD)***. Τα σχήματα AEAD αποτελούν ενιαίους μηχανισμούς που προσφέρουν ταυτόχρονα και αδιαίρετα προστασία εμπιστευτικότητας και ισχυρή επαλήθευση ακεραιότητας, με μία μόνο διέλευση (pass) πάνω από τα δεδομένα.



Εικόνα 2.1: Θεωρητικό μοντέλο λειτουργίας Authenticated Encryption with Associated Data (AEAD)

Όπως παρατηρείται στο παραπάνω διάγραμμα, ένα κρίσιμο πλεονέκτημα του AEAD για τη συγκεκριμένη υλοποίηση είναι η δυνατότητα ενσωμάτωσης Συνδεδεμένων Δεδομένων (Associated Data - AD). Το χαρακτηριστικό αυτό επιτρέπει στο σύστημα να "δέσει" κρυπτογραφικά τα μεταδεδομένα (όπως το Nonce ή έναν Counter) με το κρυπτογραφημένο αρχείο, χωρίς να τα κρυπτογραφήσει, εξασφαλίζοντας ωστόσο την απόλυτη ακεραιότητά τους. Οποιαδήποτε απόπειρα αλλοίωσης του αρχείου ή των μεταδεδομένων του, θα οδηγήσει σε αποτυχία επαλήθευσης του Authentication Tag (σφραγίδα ακεραιότητας) κατά την αποκρυπτογράφηση, και το σύστημα θα απορρίψει την ενέργεια ως κακόβουλη.

Για την κάλυψη της απαίτησης επιλογής αλγορίθμων από τον χρήστη (Cryptographic Agility), το σύστημα ενσωματώνει δύο από τα πλέον σύγχρονα AEAD primitives, τα οποία αναλύονται παρακάτω.

2.2.1 AES-GCM: Block Ciphers, Counter Mode και Hardware Acceleration

Το AES-GCM (Advanced Encryption Standard - Galois/Counter Mode) αποτελεί σήμερα το de facto βιομηχανικό πρότυπο (industry standard) για αυθεντικοποιημένη κρυπτογράφηση, το οποίο προτείνεται και από οργανισμούς όπως το NIST. Όπως αναλύθηκε στις διαλέξεις του μαθήματος, το AES είναι ένας αλγόριθμος κρυπτογράφησης τμημάτων (Block Cipher). Ωστόσο, στην κατάσταση λειτουργίας GCM, μετατρέπεται πρακτικά σε αλγόριθμο

ροής (stream cipher). Η λειτουργία του χωρίζεται σε δύο διακριτούς, αλλά παράλληλους, μηχανισμούς:

- Κρυπτογράφηση (Counter Mode - CTR): Χρησιμοποιεί το κρυπτογραφικό κλειδί και έναν μοναδικό αριθμό (Nonce/IV) για να δημιουργήσει μια ψευδοτυχαία ροή δεδομένων (keystream). Αυτή η ροή γίνεται XOR με το αρχείο (plaintext), εξασφαλίζοντας την εμπιστευτικότητα. Το πλεονέκτημα του CTR mode είναι ότι δεν απαιτεί padding (γέμισμα) των δεδομένων, διατηρώντας το μέγεθος του αρχείου ακριβώς το ίδιο.
- Αυθεντικοποίηση (Galois Message Authentication Code - GMAC): Ταυτόχρονα με την κρυπτογράφηση, τα παραγόμενα κρυπτογραφημένα δεδομένα, μαζί με τυχόν Συνδεδεμένα Δεδομένα (Associated Data), τροφοδοτούνται σε μια συνάρτηση κατακερματισμού βασισμένη σε πεδία Galois. Το αποτέλεσμα είναι ένα Authentication Tag (συνήθως 16 bytes), το οποίο αποτελεί την κρυπτογραφική απόδειξη της ακεραιότητας.

Το τεράστιο πλεονέκτημα του AES-GCM, το οποίο δικαιολογεί την επιλογή του στο παρόν σύστημα, είναι η αποδοτικότητά του (performance). Η μαθηματική δομή του επιτρέπει την πλήρη παραλληλοποίηση (parallelization) των υπολογισμών. Ακόμη πιο σημαντικό, όμως, είναι το γεγονός ότι οι σύγχρονοι επεξεργαστές (Intel, AMD, ARM) ενσωματώνουν ειδικές εντολές υλικού, γνωστές ως AES-NI (AES New Instructions).Αυτό σημαίνει ότι όταν το πρόγραμμά μας καλεί το AES-GCM (σε επίπεδα ασφαλείας είτε $\lambda = 128$ είτε $\lambda = 256$ bits), οι βαριές κρυπτογραφικές πράξεις δεν εκτελούνται στο λογισμικό, αλλά απευθείας στο Hardware (silicon) του CPU. Έτσι, το σύστημα μπορεί να κρυπτογραφεί και να ελέγχει την ακεραιότητα μεγάλων αρχείων (π.χ. video files ή μεγάλα databases) με μηδαμινή καθυστέρηση και ελάχιστη κατανάλωση πόρων.

2.2.2 ChaCha20-Poly1305: Stream Ciphers και η αρχιτεκτονική ARX

Στο πλαίσιο της παροχής κρυπτογραφικής ευελιξίας (crypto-agility), το σύστημα ενσωματώνει ως εναλλακτική επιλογή το σχήμα AEAD ChaCha20-Poly1305. Πρόκειται για έναν συνδυασμό αλγορίθμων που έχει υιοθετηθεί ευρέως στα σύγχρονα πρωτόκολλα ασφαλείας, συμπεριλαμβανομένου του TLS 1.3, αποτελώντας την ιδανική λύση για περιβάλλοντα όπου η υλική επιτάχυνση απουσιάζει.

Το σχήμα αυτό συνθέτει δύο ξεχωριστές κρυπτογραφικές λειτουργίες:

- ChaCha20 (Εμπιστευτικότητα): Αποτελεί έναν καθαρό αλγόριθμο ροής (stream cipher). Λαμβάνει ένα κλειδί (εγγενώς μήκους 256 bits) και ένα Nonce, δημιουργώντας την ψευδοτυχαία ροή (keystream) που γίνεται XOR με τα δεδομένα.
- Poly1305 (Ακεραιότητα): Λειτουργεί ως One-Time Authenticator (MAC μιας χρήσης). Λαμβάνει το παραγόμενο κρυπτογράφημα (ciphertext) και τυχόν Συνδεδεμένα Δεδομένα (AD) για να υπολογίσει το τελικό Authentication Tag επαλήθευσης.

Η αναγκαιότητα ενσωμάτωσης αυτού του αλγορίθμου πηγάζει από τους εγγενείς περιορισμούς του AES όταν εκτελείται αμιγώς σε λογισμικό (software). Οι εσωτερικές λειτουργίες του AES (όπως τα S-Boxes και ο πολλαπλασιασμός πινάκων) είναι πολύπλοκες και, αν δεν υποστηρίζονται από ειδικό υλικό (Hardware), όχι μόνο επιβραδύνουν το σύστημα, αλλά το καθιστούν και ευάλωτο σε επιθέσεις πλευρικών καναλιών (side-channel timing attacks).

Αντίθετα, η αρχιτεκτονική του ChaCha20 βασίζεται αποκλειστικά σε τρεις απλές και γρήγορες πράξεις επιπέδου επεξεργαστή: Πρόσθεση, Περιστροφή (Rotation) και XOR (μοντέλο ARX - Add-Rotate-XOR). Αυτός ο σχεδιασμός εξασφαλίζει ότι ο αλγόριθμος εκτελείται στον ίδιο ακριβώς χρόνο (constant-time execution) ανεξαρτήτως των δεδομένων, προσφέροντας ανοσία σε timing attacks και εξαιρετική ταχύτητα, ακόμη και σε παλαιότερους επεξεργαστές, κινητές συσκευές (π.χ. ARM χωρίς κρυπτογραφικό hardware) ή συστήματα IoT.

Μέσω αυτής της επιλογής, το προτεινόμενο σύστημα διασφαλίζει ότι η προστασία των αρχείων παραμένει ισχυρή και αποδοτική, ανεξάρτητα από την υποκείμενη πλατφόρμα εκτέλεσης.

2.2.3 Σύγκριση και Κρυπτογραφική Ευελιξία (Crypto-Agility)

Η παρουσία δύο διαφορετικών μηχανισμών AEAD στο σύστημα (AES-GCM και ChaCha20-Poly1305) εξυπηρετεί την αρχή της κρυπτογραφικής ευελιξίας, επιτρέποντας την προσαρμογή του λογισμικού στους πόρους του εκάστοτε περιβάλλοντος εκτέλεσης. Η τελική επιλογή από τον χρήστη πρέπει να βασίζεται κατά κύριο λόγο στη διαθεσιμότητα υλικής επιτάχυνσης (Hardware Acceleration).

Συνοπτικά, η στρατηγική επιλογής διαμορφώνεται ως εξής:

- Συστήματα με υποστήριξη AES-NI (Σύγχρονα Desktop / Server CPUs):
  - Προτεινόμενη Επιλογή: AES-GCM.
  - Αιτιολόγηση: Η ύπαρξη εξειδικευμένων εντολών στον επεξεργαστή επιτρέπει στο AES-GCM να εκτελείται με ταχύτητες που το λογισμικό δεν μπορεί να ανταγωνιστεί. Παρέχει τον βέλτιστο συνδυασμό μέγιστης απόδοσης και κορυφαίας ασφάλειας, καθιστώντας το την ιδανική επιλογή για τη μαζική κρυπτογράφηση μεγάλων αρχείων σε σύγχρονα υπολογιστικά συστήματα.
- Συστήματα χωρίς υποστήριξη AES-NI (IoT, Παλαιό Hardware, Mobile processors):
  - Προτεινόμενη Επιλογή: ChaCha20-Poly1305.
  - Αιτιολόγηση: Σε περιβάλλοντα όπου το AES αναγκάζεται να εκτελεστεί αμιγώς σε λογισμικό, παρουσιάζει δραματική πτώση απόδοσης και αυξημένο κίνδυνο επιθέσεων πλευρικών καναλιών (timing attacks). Το ChaCha20, χάρη στον ARX σχεδιασμό του, εκτελείται ταχύτερα και ασφαλέστερα υπό αυτές τις συνθήκες, διατηρώντας σταθερό χρόνο εκτέλεσης.

Μέσω αυτής της αρχιτεκτονικής προσέγγισης, το πρόγραμμα δεν δεσμεύει τον χρήστη σε μία "one-size-fits-all" λύση, αλλά του παρέχει τα εργαλεία για να βελτιστοποιήσει την ισορροπία μεταξύ ασφάλειας και ταχύτητας (Security vs. Performance trade-off), καλύπτοντας πλήρως τις απαιτήσεις της σύγχρονης μηχανικής συστημάτων.

Πίνακας 1: Σύγκριση Αλγορίθμων Authenticated Encryption (AES-GCM vs ChaCha20)

Χαρακτηριστικό	AES-GCM	ChaCha20-Poly1305
Τύπος Κρυπτογράφησης	Block Cipher (σε CTR mode)	Stream Cipher
Hardware Accel. (AES-NI)	Άριστη (Κορυφαία επιλογή)	Πολύ Καλή (Δεν αξιοποιεί ειδικό HW)
Απόδοση (Software-only)	Μέτρια / Αργή	Άριστη
Μήκος Κλειδιού	128 ή 256 bits (παραμετροποιήσιμο)	256 bits (εγγενώς)
Αντοχή σε Timing Attacks	Μέσω Hardware (AES-NI)	Εγγενής (Σχεδιασμός ARX)

2.3 Συναρτήσεις Παραγωγής Κλειδιών (KDF) και Διαχείριση Εντροπίας

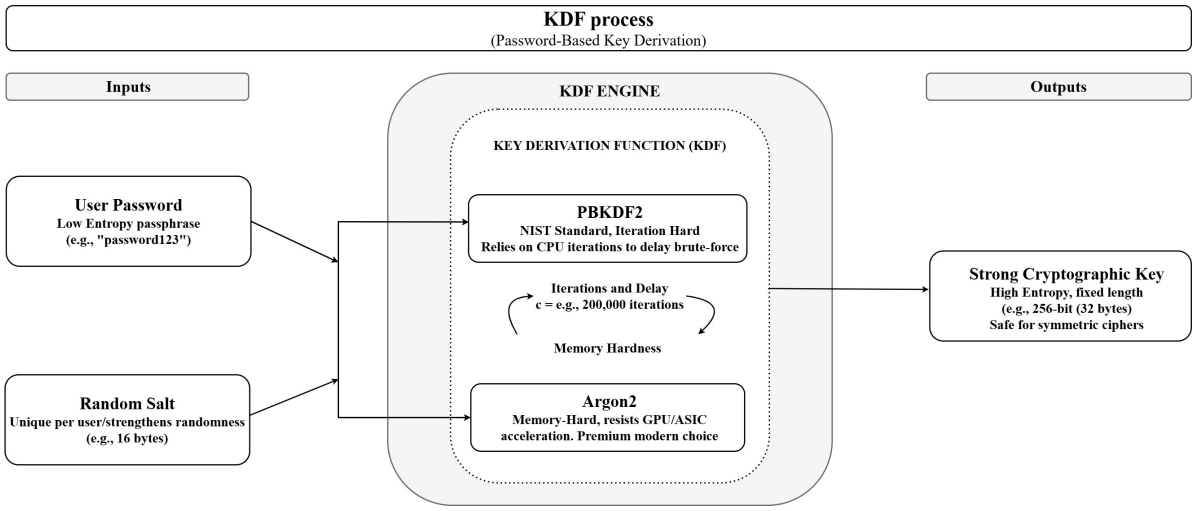
Ένα από τα πιο κρίσιμα ζητήματα στην εφαρμοσμένη κρυπτογραφία είναι η διαχείριση της εντροπίας (entropy) των κλειδιών. Οι σύγχρονοι αλγόριθμοι κρυπτογράφησης, όπως το AES-256 και το ChaCha20, απαιτούν κλειδιά μήκους 256 bits τα οποία πρέπει να είναι απολύτως τυχαία. Ωστόσο, στην πράξη (όπως απαιτείται και από τις προδιαγραφές του παρόντος συστήματος), η είσοδος συχνά προέρχεται από ένα συνθηματικό χρήστη (passphrase), το οποίο έχει εγγενώς χαμηλή εντροπία και είναι ευάλωτο σε επιθέσεις λεξικού (dictionary attacks).

Για να γεφυρωθεί αυτό το χάσμα, το σύστημα χρησιμοποιεί μια Συνάρτηση Παραγωγής Κλειδιών (Key Derivation Function - KDF). Συγκεκριμένα, αξιοποιείται ο αλγόριθμος PBKDF2 (Password-Based Key Derivation Function 2) σε συνδυασμό με την ισχυρή συνάρτηση κατακερματισμού SHA-256. Η διαδικασία αυτή, γνωστή και ως Key Stretching, επιτελεί δύο βασικές λειτουργίες:

- **Ενσωμάτωση Κρυπτογραφικού *Salt***: Το KDF δεν δέχεται μόνο το συνθηματικό, αλλά και ένα τυχαία παραγόμενο Salt (π.χ. 16 bytes). Το Salt διασφαλίζει ότι ακόμα και αν δύο χρήστες (ή δύο αρχεία) έχουν το ίδιο ακριβώς συνθηματικό, το τελικό παραγόμενο κλειδί θα είναι εντελώς διαφορετικό. Αυτό πρακτικά εκμηδενίζει τη δυνατότητα χρήσης προϋπολογισμένων πινάκων (Rainbow Tables) από έναν επιτιθέμενο.
- **Επαναληπτικότητα *(Iteration Count)***: Το KDF ρυθμίζεται να εκτελέσει τη συνάρτηση κατακερματισμού χιλιάδες φορές (π.χ. $c = 200.000$ επαναλήψεις). Αυτή η εσκεμμένη υπολογιστική καθυστέρηση (delay) είναι ανεπαίσθητη για τον νόμιμο χρήστη (διαρκεί κλάσματα του δευτερολέπτου), αλλά αποτελεί ανυπέρβλητο εμπόδιο για έναν επιτιθέμενο που προσπαθεί να δοκιμάσει εκατομμύρια πιθανά συνθηματικά ανά δευτερόλεπτο (offline brute-force attack).

Προχωρώντας ένα βήμα παραπέρα στην ενίσχυση της ασφάλειας έναντι σύγχρονων απειλών, το σύστημα προσφέρει ως εναλλακτική, πιο προηγμένη επιλογή τον αλγόριθμο Argon2. Σε αντίθεση με το PBKDF2 που εστιάζει μόνο στην επεξεργαστική καθυστέρηση

(CPU-bound), το Argon2 είναι σχεδιασμένο να είναι απαιτητικό και σε μνήμη (memory-hard). Αυτό το χαρακτηριστικό το καθιστά εξαιρετικά ανθεκτικό σε επιθέσεις ωμής βίας που εκτελούνται από εξειδικευμένο υλικό (όπως GPUs ή ASICs), αποτελώντας την κορυφαία επιλογή για την παραγωγή ισχυρών κλειδιών.



Εικόνα 2.2: Θεωρητικό μοντέλο λειτουργίας Συνάρτησης Παραγωγής Κλειδιών (KDF) με χρήση Salt και Επαναλήψεων.

Στην αρχιτεκτονική του παρόντος συστήματος, η KDF αποτελεί τον πυρήνα της δημιουργίας του Κεντρικού Κλειδιού (Master Key). Το Master Key δεν αποθηκεύεται ποτέ σε απλή μορφή, παράγεται δυναμικά κατά την είσοδο του χρήστη, και εν συνεχεία χρησιμοποιείται ως βάση (Root of Trust) για την παραγωγή των επιμέρους κλειδιών των αρχείων, επιτρέποντας την ασφαλή υλοποίηση της στρατηγικής Key Rotation.

2.4 Η Στρατηγική του Key Rotation και η αποφυγή Key/Nonce Reuse

Η θεμελιώδης αρχή της σύγχρονης εφαρμοσμένης κρυπτογραφίας επιτάσσει ότι τα κρυπτογραφικά κλειδιά πρέπει να έχουν περιορισμένο κύκλο ζωής (cryptoperiod). Στη συγκεκριμένη υλοποίηση, τίθεται η αυστηρή απαίτηση ότι κάθε αρχείο προστατεύεται με διαφορετικό κλειδί, το οποίο ανανεώνεται (Key Rotation) κάθε φορά που πραγματοποιείται πρόσβαση σε αυτό — είτε για απλό έλεγχο ακεραιότητας, είτε για πλήρη αποκρυπτογράφηση.

Αυτή η δυναμική στρατηγική επιλύει το κρισιμότερο ίσως πρόβλημα των αλγορίθμων που βασίζονται σε ροές δεδομένων (stream ciphers), όπως το ChaCha20, αλλά και των block ciphers που λειτουργούν σε Counter Mode (όπως το AES-GCM): το Key Reuse (Επαναχρησιμοποίηση Κλειδιού) και το Nonce Collision.

Όπως αναλύεται στη θεωρία, αν το ίδιο κλειδί (K) και το ίδιο Nonce (N) χρησιμοποιηθούν για να κρυπτογραφήσουν δύο διαφορετικά αρχεία (ή δύο διαφορετικές εκδόσεις του ίδιου αρχείου), παράγεται ακριβώς η ίδια ψευδοτυχαία ροή (keystream).

Αν ένας επιτιθέμενος υποκλέψει τα δύο κρυπτογραφήματα (C_1 και C_2) και εφαρμόσει την πράξη XOR μεταξύ τους, η κρυπτογραφική ροή ακυρώνεται και αποκαλύπτεται το XOR των δύο αρχικών κειμένων ($P_1 \oplus P_2$), καταστρέφοντας πλήρως την εμπιστευτικότητα.

Για την αποτροπή αυτής της ευπάθειας, το σύστημα υλοποιεί έναν μηχανισμό διαχείρισης Κατάστασης (State Management). Η διαδικασία λειτουργεί ως εξής:

- Κατά την αρχική κρυπτογράφηση ενός αρχείου, παράγεται ένα μοναδικό File Key (FK_1) μέσω του KDF, χρησιμοποιώντας το Master Key και έναν αρχικό μετρητή (Counter/State).
- Όταν ο χρήστης ζητήσει αποκρυπτογράφηση ή επαλήθευση ακεραιότητας, το σύστημα χρησιμοποιεί το FK_1 για να ολοκληρώσει τη λειτουργία.
- Αμέσως μετά την επιτυχή εκτέλεση της λειτουργίας, το σύστημα αναγκαστικά αυξάνει τον μετρητή (state increment) και παράγει ένα νέο κλειδί (FK_2). Το αρχείο επανακρυπτογραφείται ή "επανασφραγίζεται" (re-tagged) με το νέο κλειδί.

Η προσέγγιση αυτή προσφέρει ένα ισχυρό μοντέλο που προσομοιάζει την έννοια του Forward Secrecy. Ακόμα και αν ένας επιτιθέμενος καταφέρει να ανακτήσει το κλειδί ενός αρχείου σε μια δεδομένη χρονική στιγμή, αυτό το κλειδί καθίσταται άχρηστο για τις επόμενες ή τις προηγούμενες εκδόσεις του ίδιου αρχείου. Έτσι, περιορίζεται δραματικά μια πιθανή διαρροή.

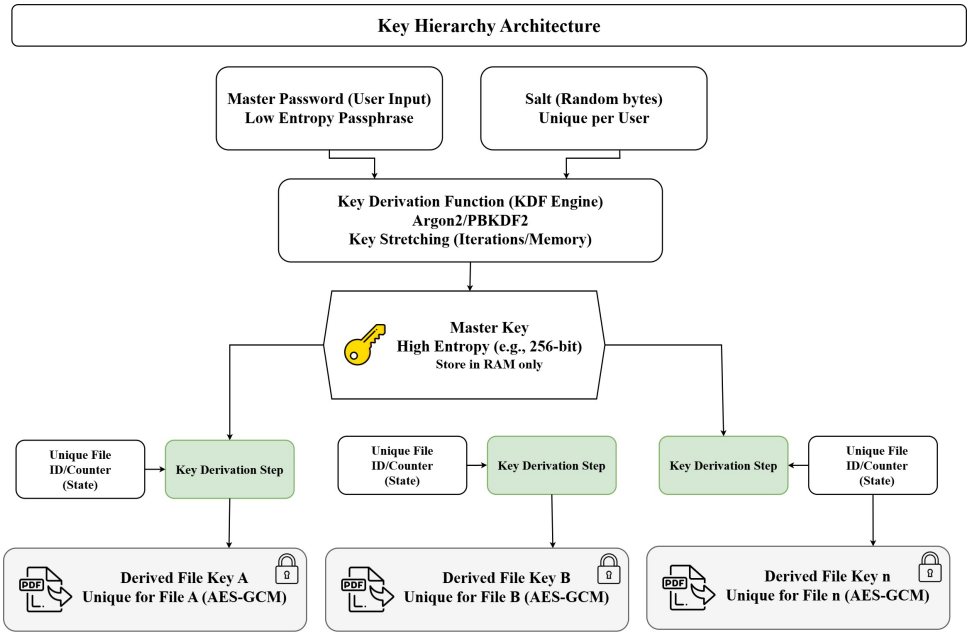
3 Κεφάλαιο 3: Σχεδιασμός και Αρχιτεκτονική Συστήματος

Στο παρόν κεφάλαιο παρουσιάζεται ο πρακτικός σχεδιασμός της εφαρμογής, γεφυρώνοντας το θεωρητικό υπόβαθρο με την τελική υλοποίηση. Στόχος είναι να καθοριστεί η αρχιτεκτονική που ικανοποιεί τις βασικές απαιτήσεις της εργασίας, εστιάζοντας κυρίως στην προστασία κάθε αρχείου με διαφορετικό κλειδί και στην ασφαλή διαχείρισή τους. Συνοπτικά, στις επόμενες ενότητες εξετάζονται τα εξής:

- Η ιεραρχία των κλειδιών (Key Hierarchy): Πώς το αρχικό συνθηματικό του χρήστη (Master Password) χρησιμοποιείται για να παραχθούν με ασφάλεια μοναδικά κλειδιά για κάθε ξεχωριστό αρχείο.
- Ο μηχανισμός Key Rotation: Ποιοι είναι οι κανόνες που «πυροδοτούν» την αλλαγή του κλειδιού ενός αρχείου (π.χ. κατά τον έλεγχο ακεραιότητας).
- Η συνολική ροή (Workflow): Η οπτικοποίηση των βημάτων του συστήματος και ο τρόπος με τον οποίο αποθηκεύονται οι απαραίτητες πληροφορίες (μεταδεδομένα), ώστε το πρόγραμμα να ξέρει πώς να αποκρυπτογραφήσει το αρχείο στο μέλλον.

3.1 Αρχιτεκτονική Ιεραρχίας Κλειδιών (Key Hierarchy)

Η απαίτηση της εργασίας είναι ξεκάθαρη: κάθε αρχείο πρέπει να προστατεύεται με διαφορετικό κλειδί. Για να επιτευχθεί αυτό χωρίς ο χρήστης να χρειάζεται να θυμάται δεκάδες διαφορετικούς κωδικούς, το σύστημα υλοποιεί μια δενδρική δομή (ιεραρχία) κλειδιών, η οποία αποτυπώνεται οπτικά στην Εικόνα 3.1



Εικόνα 3.1: Δενδρική αναπαράσταση της ιεραρχίας κλειδιών.

3.1.1 Root Key (Master Password) και Entropy

Το σημείο εκκίνησης (η "ρίζα" της ιεραρχίας) είναι το συνθηματικό (Master Password) που εισάγει ο χρήστης. Όπως αναλύθηκε στο θεωρητικό μέρος, το συνθηματικό αυτό δεν χρησιμοποιείται ποτέ απευθείας για κρυπτογράφηση. Αντίθετα, περνάει μέσα από τη συνάρτηση KDF (PBKDF2), συνδυάζεται με ένα κρυπτογραφικό άλας (Salt) και μετατρέπεται στο κεντρικό Master Key. Αυτό το ισχυρό κλειδί δεν αποθηκεύεται πουθενά μόνιμα, παρά μόνο προσωρινά στη μνήμη του προγράμματος κατά την εκτέλεσή του.

3.1.2 Derived File Keys: Η απομόνωση των κλειδιών

Αφού το σύστημα έχει παράγει το κεντρικό Master Key, το χρησιμοποιεί ως βάση για να "γεννήσει" τα επιμέρους κλειδιά των αρχείων (File Keys). Η διαδικασία έχει ως εξής:

- Για κάθε νέο αρχείο (ή νέα ενέργεια στο αρχείο), το σύστημα συνδυάζει το Master Key με έναν μοναδικό μετρητή (counter).
- Το αποτέλεσμα αυτού του συνδυασμού δημιουργεί ένα νέο, εντελώς ανεξάρτητο File Key.
- Αυτό το File Key αναλαμβάνει την κρυπτογράφηση και την παραγωγή του MAC (Tag) αποκλειστικά για το συγκεκριμένο αρχείο.

Η προσέγγιση αυτή προσφέρει απόλυτη απομόνωση. Αν, σε ένα υποθετικό σενάριο, διαρρεύσει το κλειδί ενός συγκεκριμένου αρχείου, όλα τα υπόλοιπα αρχεία του χρήστη παραμένουν 100% ασφαλή, καθώς το καθένα προστατεύεται από το δικό του, ανεξάρτητο File Key.

3.2 Σχεδιασμός του Μηχανισμού Key Rotation *(State Management)*

Ένας από τους βασικούς πυλώνες ασφάλειας της εφαρμογής είναι η αποτροπή της επαναχρησιμοποίησης του ίδιου κρυπτογραφικού κλειδιού (και του αντίστοιχου Nonce) για το ίδιο αρχείο. Αν ένα κλειδί χρησιμοποιηθεί πολλές φορές για να κρυπτογραφήσει διαφορετικές εκδόσεις του ίδιου αρχείου, αυξάνεται ο κίνδυνος κρυπτογραφικής ανάλυσης. Για να αποφευχθεί αυτό, σχεδιάστηκε ένας μηχανισμός Key Rotation (Ανακύκλωση Κλειδιών), ο οποίος βασίζεται στη διαχείριση καταστάσεων (State Management).

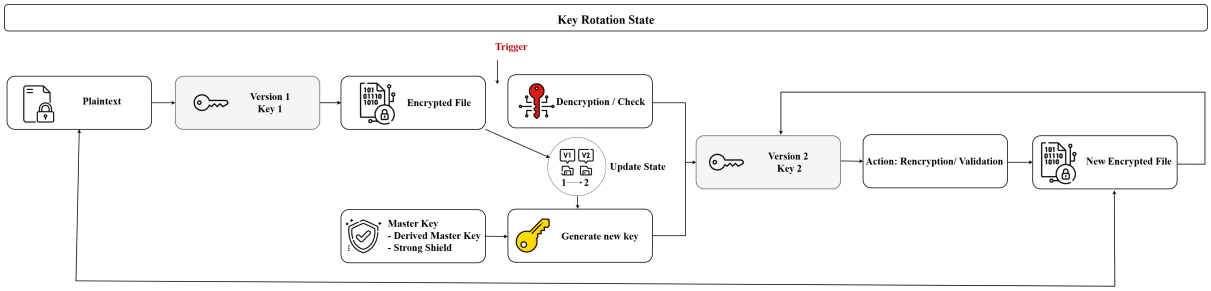
3.2.1 Trigger Events: Rotation κατά το Decryption vs Integrity Check

Η εκφώνηση απαιτεί η αλλαγή του κλειδιού να πραγματοποιείται όταν εκτελούνται συγκεκριμένες ενέργειες. Το σύστημα έχει σχεδιαστεί να αναγνωρίζει (trigger) δύο βασικά γεγονότα:

- **Μόνο Έλεγχος Ακεραιότητας *(Integrity Check)***: Το σύστημα διαβάζει το αρχείο, επαληθεύει μέσω του MAC (Tag) ότι δεν έχει αλλοιωθεί και, εφόσον ο έλεγχος είναι επιτυχής, το "ξανασφραγίζει" αυτόματα χρησιμοποιώντας ένα νέο κλειδί.

- **Αποκρυπτογράφηση & Έλεγχος *(Decryption& Integrity)***: Το πρόγραμμα επαληθεύει την ακεραιότητα και αποκρυπτογραφεί τα δεδομένα. Όταν τα δεδομένα κρυπτογραφηθούν ξανά για να αποθηκευτούν, το σύστημα χρησιμοποιεί ένα νέο κλειδί.

Η εναλλαγή του κλειδιού επιτυγχάνεται πρακτικά με την αύξηση ενός εσωτερικού μετρητή κατάστασης (state counter). Κάθε φορά που γίνεται μια από τις παραπάνω ενέργειες, ο μετρητής αυξάνεται ($counter + 1$) και το KDF παράγει ένα ολοκαίνουργιο File Key.



Εικόνα 3.2: Διάγραμμα κατάστασης ανακύκλωσης κλειδιών (Key Rotation State) και συνθήκες πυροδότησης.

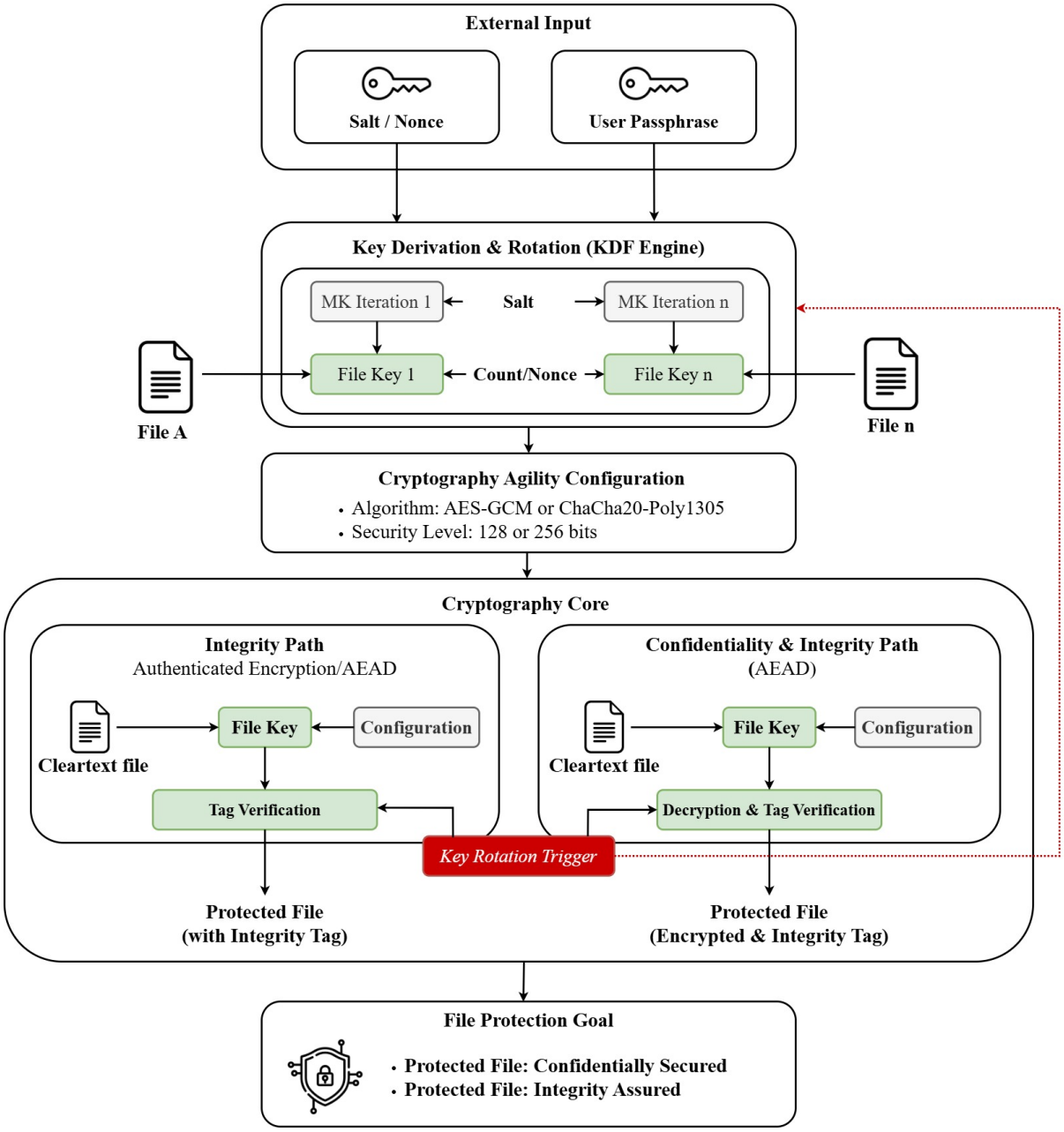
3.2.2 Forward Secrecy: Προστατεύοντας το παρελθόν

Η βασική αρχιτεκτονική αξία αυτής της συχνής αλλαγής κλειδιών είναι η ιδιότητα του Forward Secrecy (Μελλοντική Μυστικότητα). Αν ένας επιτιθέμενος καταφέρει σε μια δεδομένη χρονική στιγμή να υποκλέψει το τρέχον κλειδί ενός αρχείου, θα αποκτήσει πρόσβαση μόνο στη συγκεκριμένη έκδοση. Οποιεσδήποτε παλαιότερες (ή και μελλοντικές) καταστάσεις του ίδιου αρχείου προστατεύονται από εντελώς διαφορετικά κλειδιά, τα οποία το σύστημα έχει ήδη καταστρέψει από τη μνήμη του, περιορίζοντας έτσι δραστικά τη ζημιά.

3.3 Διάγραμμα Ροής Λειτουργίας (Workflow Diagram)

Για την καλύτερη κατανόηση της αρχιτεκτονικής, η εσωτερική λειτουργία του συστήματος (backend logic) αποτυπώνεται στο Διάγραμμα Ροής Δεδομένων (Data Flow Diagram) του Σχήματος 3.3. Το διάγραμμα απεικονίζει τη διαδρομή των πληροφοριών: από την αρχική εισαγωγή του αρχείου και του συνθηματικού από τον χρήστη, στη δημιουργία του Master Key μέσω της KDF, μέχρι την εφαρμογή του AEAD πρωτοκόλλου και την τελική παραγωγή του προστατευμένου αρχείου.

Η οπτικοποίηση αυτή καταδεικνύει την αυστηρή αλληλουχία των κρυπτογραφικών βημάτων και τον καθαρό διαχωρισμό μεταξύ της διαχείρισης των κλειδιών (Key Management Layer) και της επεξεργασίας των δεδομένων (Data Processing Layer).



Εικόνα 3.3: Αρχιτεκτονική του συστήματος προστασίας αρχείων και διάγραμμα ροής δεδομένων (Data Flow Diagram).

3.4 Διαχείριση Μεταδεδομένων (Metadata Headers)

Μια από τις μεγαλύτερες προκλήσεις στον σχεδιασμό του συστήματος είναι η διατήρηση της "μνήμης" των κρυπτογραφικών παραμέτρων. Για να μπορέσει το πρόγραμμα να αποκρυπτογραφήσει επιτυχώς ένα αρχείο στο μέλλον, πρέπει να γνωρίζει ακριβώς ποιο Salt, ποιο Nonce και ποιον μετρητή (Rotation State) χρησιμοποίησε την τελευταία φορά, χωρίς προφανώς να αποθηκεύει το ίδιο το κλειδί.

Η αρχιτεκτονική επιλύει αυτό το πρόβλημα καθιστώντας κάθε προστατευόμενο αρχείο απολύτως αυτόνομο (self-contained). Αυτό επιτυγχάνεται με την ενσωμάτωση μιας μη κρυπτογραφημένης "κεφαλίδας" (Metadata Header) στην αρχή του αρχείου. Η τελική ανατομία του αποθηκευμένου αρχείου στο δίσκο διαμορφώνεται ως εξής:

1 Μη Κρυπτογραφημένο Header (Απλό Κείμενο): * Salt (π.χ. 16 bytes): Απαραίτητο για να τροφοδοτήσει την KDF.

- Nonce/IV (π.χ. 12 bytes): Απαραίτητο για την αρχικοποίηση του AEAD.
- State Counter (π.χ. 4 bytes): Υποδεικνύει σε ποιο "rotation" βρίσκεται το αρχείο, ώστε να παραχθεί το σωστό File Key.

2 Κρυπτογραφημένο Payload:

- Ακολουθούν τα πραγματικά κρυπτογραφημένα δεδομένα (Ciphertext).
- Στο τέλος προσαρτάται το Authentication Tag (MAC) για την επαλήθευση της ακεραιότητας όλου του πακέτου.

Τα δεδομένα της κεφαλίδας είναι δημόσια και λειτουργούν ως Συνδεδεμένα Δεδομένα (Associated Data - AD) στον αλγόριθμο AEAD. Οποιαδήποτε απόπειρα αλλοίωσής τους από κάποιον επιτιθέμενο θα εντοπιστεί άμεσα κατά τον έλεγχο του Authentication Tag, αποτρέποντας επιθέσεις υποβάθμισης (downgrade attacks) ή αλλοίωσης του rotation state.

4 Κεφάλαιο 4: Υλοποίηση και Ανάλυση Κώδικα

Στο προηγούμενο κεφάλαιο καθορίστηκε η αρχιτεκτονική και ο μηχανισμός λειτουργίας του συστήματος. Στο παρόν κεφάλαιο, η θεωρητική σχεδίαση μεταβαίνει στο στάδιο της πρακτικής εφαρμογής, παρουσιάζοντας την υλοποίηση του λογισμικού προστασίας αρχείων. Στόχος αυτού του κεφαλαίου είναι να αναδείξει τον τρόπο με τον οποίο οι αυστηρές κρυπτογραφικές απαιτήσεις (όπως η απομόνωση κλειδιών, το key rotation και το crypto-agility) μεταφράζονται σε ασφαλή, λειτουργικό και αποδοτικό κώδικα.

Η ανάλυση που ακολουθεί δεν αποτελεί μια απλή παράθεση του πηγαίου κώδικα (ο οποίος παρατίθεται αυτούσιος στο Παράρτημα), αλλά εστιάζει στα σημαντικότερα κομμάτια της υλοποίησης. Εξετάζονται οι βιβλιοθήκες που επιλέχθηκαν, η αντικειμενοστραφής (Object-Oriented) προσέγγιση της βασικής κλάσης, ο τρόπος υλοποίησης της δυναμικής παραμετροποίησης από τον χρήστη, καθώς και οι μηχανισμοί χειρισμού σφαλμάτων για την αποτροπή διαρροών πληροφορίας (information leakage) κατά την εκτέλεση.

4.1 Περιβάλλον Ανάπτυξης και Βιβλιοθήκες (Python Cryptography)

Για την υλοποίηση του συστήματος επιλέχθηκε η γλώσσα προγραμματισμού Python (v3.x), λόγω της ισχυρής υποστήριξης σε κρυπτογραφικές βιβλιοθήκες και της ικανότητας για ταχεία ανάπτυξη (rapid prototyping). Ωστόσο, η καρδιά της εφαρμογής βασίζεται σε εξειδικευμένα εργαλεία που διασφαλίζουν τη χαμηλού επιπέδου (low-level) διαχείριση των αλγορίθμων:

- ***Pyca/Cryptography (Hazmat Layer[1])***: Χρησιμοποιήθηκε το επίπεδο Hazardous Materials (Hazmat) της βιβλιοθήκης. Η επιλογή αυτή έγινε συνειδητά για να έχουμε άμεσο έλεγχο των κρυπτογραφικών πρωτογόνων (primitives) όπως οι AES-GCM, ChaCha20-Poly1305 και HMAC-SHA256, αποφεύγοντας "μαύρα κουτιά" που μπορεί να κρύβουν αδυναμίες.
- ***Argon2id (argon2-cffi)***: Για τη διαδικασία παραγωγής κλειδιών (Key Derivation), επιλέχθηκε ο αλγόριθμος Argon2id. Πρόκειται για τον νικητή του Password Hashing Competition (PHC), ο οποίος προσφέρει ανώτερη προστασία έναντι επιθέσεων μέσω GPU (memory-hard function) και προστασία από side-channel attacks (timing attacks).
- ***OS/Getpass***: Για τη διαχείριση του συστήματος αρχείων και την ασφαλή είσοδο κωδικών από τον χρήστη (χωρίς εμφάνιση των χαρακτήρων στην οθόνη), διασφαλίζοντας την τοπική ασφάλεια κατά τη χρήση.

Η επιλογή αυτών των βιβλιοθηκών δεν έγινε μόνο με γνώμονα τη λειτουργικότητα, αλλά κυρίως για να υποστηρίξει το Crypto-Agility της εφαρμογής, επιτρέποντας την εναλλαγή μεταξύ διαφορετικών αλγορίθμων με κοινή λογική διαχείρισης κλειδιών.

[1] Το Hazmat layer παρέχει άμεση πρόσβαση σε low-level κρυπτογραφικούς πρωτογενείς αλγορίθμους (primitives). Περισσότερες πληροφορίες: https://cryptography.io/en/latest/hazmat/primitives/

4.2 Ανάλυση της Κλάσης Προστασίας Αρχείων

Στην ενότητα αυτή αναλύεται η κλάση SecureFileProtector, η οποία αποτελεί την υλοποίηση της αρχιτεκτονικής που περιγράφηκε στο Κεφάλαιο 3. Η ανάλυση επικεντρώνεται στα τρία κρίσιμα υποσυστήματα (components) που διασφαλίζουν την ασφάλεια των δεδομένων: την παραγωγή κλειδιών, τη διαδικασία κρυπτογράφησης και τον μηχανισμό ανανέωσης κλειδιών (Key Rotation).

4.2.1 Παραγωγή Κρυπτογραφικού Υλικού (`_derive_master_key`)

Η πιο κρίσιμη μέθοδος της κλάσης είναι η `_derive_master_key`. Η συγκεκριμένη συνάρτηση υλοποιεί την παραγωγή κλειδιών μέσω του ***Argon2id***, μετατρέποντας το password του χρήστη σε ένα κρυπτογραφικά ισχυρό κλειδί. Η χρήση τυχαίου Salt 128-bit για κάθε αρχείο διασφαλίζει ότι ακόμα και αν δύο αρχεία έχουν τον ίδιο κωδικό πρόσβασης, τα τελικά κλειδιά κρυπτογράφησης θα είναι εντελώς διαφορετικά.

```
def _derive_master_key(self, passphrase: str, salt: bytes) ->
   bytes:
    master_key = low_level.hash_secret_raw(
        secret=passphrase.encode(),
        salt=salt,
        time_cost=3,
        memory_cost=65536,
        parallelism=4,
        hash_len=32, # Output 256 bits
        type=low_level.Type.ID
    )
    return master_key[:self.key_size // 8]
```

Listing 1: Υλοποίηση Key Derivation Function (KDF) με Argon2id

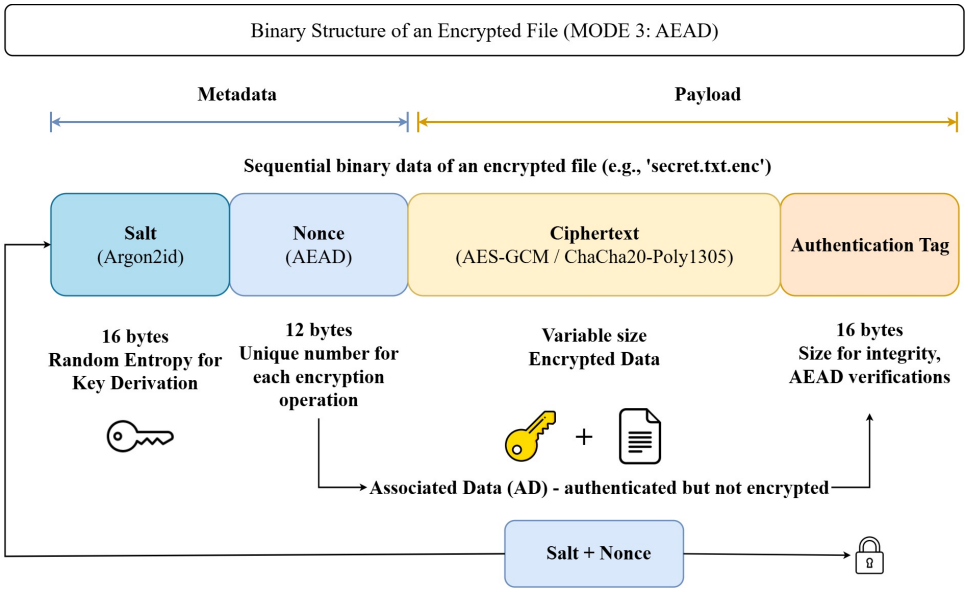
Στην υλοποίηση της μεθόδου, επιλέχθηκε η παραγωγή ενός σταθερού κρυπτογραφικού υλικού 256 bits, το οποίο στη συνέχεια περικόπτεται (truncated) στο επιθυμητό μέγεθος κλειδιού (`selfkey_size`). Η προσέγγιση αυτή διασφαλίζει ότι η υπολογιστική πολυπλοκότητα του αλγορίθμου παραμένει σταθερά υψηλή, ανεξάρτητα από το επιλεγμένο επίπεδο ασφαλείας, αποτρέποντας επιθέσεις που θα εκμεταλλεύονταν πιθανή μείωση των παραμέτρων του KDF.

4.2.2 Διαδικασία Κρυπτογράφησης και Δομή Metadata

Σε αυτό το component, η θεωρητική σχεδίαση μεταφράζεται σε λειτουργικό κώδικα μέσω των μεθόδων `protect_file` και `unprotect_file`. Η υλοποίηση διασφαλίζει ότι κάθε κρυπτογραφημένο αρχείο φέρει ένα πρόθεμα (prefix) από *Metadata*, το οποίο περιλαμβάνει το *Salt* και το *Nonce*. Η προσέγγιση αυτή καθιστά το αρχείο «αυτόνομο» (self-contained), επιτρέποντας την αποκρυπτογράφηση του μόνο με τη γνώση του συνθηματικού, χωρίς την ανάγκη εξωτερικής βάσης δεδομένων για την αποθήκευση των παραμέτρων.

Όπως απεικονίζεται στην Εικόνα 4.1, η δομή του αρχείου ακολουθεί μια αυστηρή σειριακή διάταξη. Τα πρώτα bytes καταλαμβάνονται από το *Salt* (απαραίτητο για τον KDF) και το *Nonce* (απαραίτητο για τον αλγόριθμο κρυπτογράφησης), ακολουθούμενα από το κυρίως

σώμα του κρυπτογραφημένου περιεχομένου (*Ciphertext*) και, στις περιπτώσεις AEAD, το *Authentication Tag*.



Εικόνα 4.1: Σχηματική αναπαράσταση της σειριακής δομής των Metadata και του Ciphertext στο κρυπτογραφημένο αρχείο.

4.2.3 Μηχανισμός Ανανέωσης Κλειδιών (Key Rotation)

Το τρίτο key component, η μέθοδος `rotate_key`, υλοποιεί την απαίτηση για δυναμική ανανέωση της ασφάλειας. Η μέθοδος αυτή εκτελεί μια ατομική λειτουργία: αποκρυπτογραφεί τα δεδομένα με το υπάρχον κλειδί, παράγει νέο τυχαίο Salt και, μέσω του KDF, εκτελεί επανακρυπτογράφηση με το νέο κλειδί. Η διαδικασία αυτή εγγυάται το "Key Freshness" και περιορίζει την έκθεση των δεδομένων σε περίπτωση παραβίασης ενός παλαιού κωδικού.

4.3 Υλοποίηση Δυναμικής Παραμετροποίησης (Crypto-Agility)

Σε αντίθεση με στατικά κρυπτογραφικά εργαλεία, η παρούσα υλοποίηση ενσωματώνει την αρχή του Crypto-Agility. Αυτό επιτυγχάνεται μέσω του δυναμικού ορισμού των αλγορίθμων κατά το instantiation της κλάσης, επιτρέποντας στο λογισμικό να προσαρμόζεται στις ανάγκες του χρήστη ή στις προδιαγραφές του hardware.

4.3.1 Επιλογή Αλγορίθμου: Υλοποίηση AES-GCM vs ChaCha20-Poly1305

Η εφαρμογή προσφέρει δύο κορυφαίες επιλογές AEAD (Authenticated Encryption with Associated Data). Η διαφοροποίηση στον κώδικα έγκειται στην ενεργοποίηση του αντίστοιχου primitive από το Hazmat layer:

- ***AES-GCM***: Επιλέγεται για συστήματα που υποστηρίζουν hardware acceleration (AES-NI instructions), προσφέροντας μέγιστη ταχύτητα σε επίπεδο επεξεργαστή.
- ***ChaCha20-Poly1305***: Αποτελεί την εναλλακτική λύση για περιβάλλοντα software-only, εξασφαλίζοντας υψηλή απόδοση χωρίς την ανάγκη ειδικού hardware.

4.3.2 Επιλογή Παραμέτρου Ασφαλείας: Στόχοι $\lambda = 128$ και $\lambda = 256$ bits

Το σύστημα επιτρέπει τον καθορισμό του επιπέδου ασφαλείας σε 128 ή 256 bits. Όπως αναλύθηκε στην ενότητα 4.2.1, η επιλογή αυτή επηρεάζει το τελικό στάδιο του Key Derivation (Truncation), διασφαλίζοντας ότι το master key έχει το κατάλληλο μήκος για τον επιλεγμένο αλγόριθμο, τηρώντας τις σύγχρονες απαιτήσεις για ανθεκτικότητα έναντι brute-force επιθέσεων.

4.4 Χειρισμός Εξαιρέσεων και Ασφάλεια Λαθών

Η ασφάλεια ενός κρυπτογραφικού συστήματος κρίνεται από την ανθεκτικότητά του σε μη κανονικές συνθήκες λειτουργίας. Η υλοποίηση αποφεύγει τη χρήση γενικών εξαιρέσεων (Generic Exceptions), καθώς αυτές μπορούν να οδηγήσουν σε *Information Leakage*. Αντίθετα, διαχειρίζεται εξειδικευμένα σφάλματα για να προστατεύσει την ακεραιότητα της διαδικασίας.

4.4.1 Προστασία Ακεραιότητας και Αυθεντικοποίησης

Η εφαρμογή αναγνωρίζει συγκεκριμένα τις εξαιρέσεις `InvalidTag` (για AEAD) και έχουμε το `InvalidSignature` (για HMAC). Αυτό είναι ζωτικής σημασίας για την αποτροπή επιθέσεων όπως οι *Padding Oracle*, καθώς εάν το authentication tag δεν επαληθευτεί, η διαδικασία διακόπτεται αμέσως πριν την παραγωγή οποιουδήποτε plaintext.

Στην περίπτωση του HMAC (Mode 2), όπως φαίνεται στο Σχήμα 4.1, η αποτυχία επαλήθευσης της υπογραφής λόγω λανθασμένου κωδικού ή αλλοίωσης δεδομένων οδηγεί σε ασφαλή τερματισμό.

```
[!] Select Mode:
 1. Confidentiality (AES-CTR)
 2. Integrity (HMAC-SHA256)
 3. Confidentiality & Integrity (Combined)
 4. Rotate Key / Change Password

> 2

Choose Action: [1] Protect/Sign [2] Unprotect/Verify

> 2

[!] Security Level: [1] 128-bit [2] 256-bit

> 2

Enter file path: secret.txt
Password:
[!] CRITICAL: Signature Verification Failed! The file has been tampered with or the password is wrong.
(venv) PS C:\Users\katsi\Documents\Projects\secure-file-rotation-python\src>
```

Εικόνα 4.2: Ασφαλής διαχείριση του `InvalidSignature` exception σε HMAC-SHA256 Mode.

Αντίστοιχα, στα Combined AEAD Modes (AES-GCM / ChaCha20), το σύστημα εγκλωβίζει την αποτυχία του tag (Σχήμα 4.2), διασφαλίζοντας την ακεραιότητα του σχήματος.

```
 3. Confidentiality & Integrity (Combined)
 4. Rotate Key / Change Password

> 3

[!] Select Algorithm:
 1. AES-GCM
 2. ChaCha20-Poly1305

> 2

Choose Action: [1] Protect/Sign [2] Unprotect/Verify

> 2

[!] Security Level: [1] 128-bit [2] 256-bit

> 2

Enter file path: secret.txt
Password:
[!] Authentication Failed: Wrong password or corrupted metadata.
(venv) PS C:\Users\katsi\Documents\Projects\secure-file-rotation-python\src>
```

Εικόνα 4.3: Αποτυχία αυθεντικοποίησης (InvalidTag) σε AEAD σχήμα λόγω λανθασμένου password.

4.4.2 Διαχείριση Σφαλμάτων κατά την Αποκρυπτογράφηση και το Key Rotation

Εκτός από τους ελέγχους ακεραιότητας, το σύστημα θωρακίζεται και έναντι σφαλμάτων παραμετροποίησης. Για παράδειγμα, στο AES-CTR (Mode 1), το σύστημα ελέγχει την ορθότητα των metadata (Nonce size), εμποδίζοντας την εκτέλεση με λανθασμένες παραμέτρους (Εικόνα 4.4).

```
[!] Select Mode:
 1. Confidentiality (AES-CTR)
 2. Integrity (HMAC-SHA256)
 3. Confidentiality & Integrity (Combined)
 4. Rotate Key / Change Password

> 1

Choose Action: [1] Protect/Sign [2] Unprotect/Verify

> 2

[!] Security Level: [1] 128-bit [2] 256-bit

> 2

Enter file path: secret.txt
Password:
[!] System Error during CTR decryption: Invalid nonce size (13) for CTR.
(venv) PS C:\Users\katsi\Documents\Projects\secure-file-rotation-python\src>
```

Εικόνα 4.4: Εντοπισμός και διαχείριση σφάλματος μεγέθους Nonce κατά τη διαδικασία αποκρυπτογράφησης σε AES-CTR Mode.

Τέλος, η σύνθετη λειτουργία του Key Rotation (Mode 4) απαιτεί την επιτυχή επαλήθευση του αρχείου με το παλαιό κλειδί πριν την έκδοση νέου. Όπως απεικονίζεται στο Σχήμα 4.4, αν ο παλαιός κωδικός είναι λανθασμένος, η διαδικασία διακόπτεται ατομικά (atomically), προστατεύοντας το αρχείο από καταστροφή.

```
[!] Select Mode:
 1. Confidentiality (AES-CTR)
 2. Integrity (HMAC-SHA256)
 3. Confidentiality & Integrity (Combined)
 4. Rotate Key / Change Password

Enter selection (1-4): 4

[!] Select Algorithm:
 1. AES-GCM
 2. ChaCha20-Poly1305

Enter selection (1-2): 2

[!] Security Level:
 [1] 128-bit
 [2] 256-bit

Enter selection (1-2): 2

Enter file path: secret.txt
Old Password:
New Password:
[!] Rotation Failed: The old password is incorrect or the file is not a valid encrypted file.
(venv) PS C:\Users\katsi\Documents\Projects\secure-file-rotation-python\src>
```

Εικόνα 4.5: Αποτυχία της διαδικασίας Key Rotation λόγω λανθασμένου παλαιού κωδικού.

Με την ολοκλήρωση της ανάλυσης των κρυπτογραφικών συνιστωσών και των μηχανισμών ασφαλείας, καθίσταται σαφές ότι η αρχιτεκτονική του συστήματος έχει θωρακιστεί τόσο σε επίπεδο αλγορίθμων όσο και σε επίπεδο διαχείρισης σφαλμάτων. Στο επόμενο κεφάλαιο, η έμφαση μετατοπίζεται στην πειραματική αξιολόγηση του λογισμικού, όπου μέσω εκτενών δοκιμών (Testing) και σεναρίων χρήσης, θα επαληθευτεί η ορθή λειτουργία και η αξιοπιστία της εφαρμογής σε πραγματικό περιβάλλον εκτέλεσης.

5 Κεφάλαιο 5: Αξιολόγηση και Δοκιμές (Testing)

Σε αυτό το κεφάλαιο και μετά την ολοκλήρωση της υλοποίησης, κρίθηκε αναγκαία η διεξαγωγή συστηματικών δοκιμών για την επαλήθευση της ορθότητας και της αξιοπιστίας του συστήματος. Το παρόν κεφάλαιο παρουσιάζει τα αποτελέσματα της αξιολόγησης, καλύπτοντας τόσο σενάρια ορθής λειτουργίας (Positive Testing) όσο και σενάρια ανίχνευσης επιθέσεων ή σφαλμάτων. Στόχος των δοκιμών είναι να αποδειχθεί ότι οι κρυπτογραφικοί μηχανισμοί ανταποκρίνονται στις απαιτήσεις για εμπιστευτικότητα και ακεραιότητα, διασφαλίζοντας παράλληλα τη σωστή διαχείριση των μεταδεδομένων και την επιτυχή εκτέλεση της περιστροφής κλειδιών (Key Rotation).

5.1 Σενάρια Ορθής Λειτουργίας (Positive Testing)

Στην ενότητα αυτή εξετάζεται η βασική λειτουργικότητα του συστήματος υπό κανονικές συνθήκες. Στόχος είναι η επιβεβαίωση ότι η εφαρμογή μπορεί να κρυπτογραφήσει ένα αρχείο και στη συνέχεια να ανακτήσει το αρχικό περιεχόμενο (plaintext) χωρίς καμία αλλοίωση.

Όπως φαίνεται στο Σχήμα 5.1, η διαδικασία ξεκινά με την επιλογή του αρχείου `secret.txt` και την εφαρμογή προστασίας Mode 3 (AES-GCM). Το σύστημα παράγει το κρυπτογραφημένο αρχείο `secret.txt.enc`, το οποίο περιέχει τα σειριακά metadata (Salt, Nonce) και το κρυπτογραφημένο περιεχόμενο.

```
[!] Security Level:
 [1] 128-bit
 [2] 256-bit

Enter selection (1-2): 2

Enter file path: secret.txt
Password:
[+] Mode 3 (AES-GCM) complete: secret.txt.enc (Metadata Integrity Secured)
[+] Original file deleted for security.
(venv) PS C:\Users\katsi\Documents\Projects\secure-file-rotation-python\src>
```

Εικόνα 5.1: Επιτυχής κρυπτογράφηση αρχείου και διαγραφή του αρχικού για λόγους ασφαλείας.

Στη συνέχεια, εκτελείται η διαδικασία αποκάλυψης (*Unprotect*). Όπως απεικονίζεται στο Σχήμα 5.2, με την εισαγωγή του ορθού κωδικού πρόσβασης, το σύστημα επαληθεύει την ακεραιότητα, ανακτά το αρχικό περιεχόμενο στο αρχείο `secret.txt.restored` και ενεργοποιεί αυτόματα τον μηχανισμό *Key Rotation*. Η διαδικασία αυτή διασφαλίζει ότι το κρυπτογραφημένο αρχείο αποκτά νέο *Salt* και κλειδί μετά από κάθε επιτυχή ανάγνωση.

```
Enter file path: secret.txt.enc
Password:
[+] Integrity Verified! Content restored to: secret.txt.restored
[*] Triggering automatic key rotation...
[+] Mode 3 (AES-GCM) complete: secret.txt.enc (Metadata Integrity Secured)
[+] Key automatically rotated successfully.
```

Εικόνα 5.2: Επιτυχής επαλήθευση ακεραιότητας, ανάκτηση δεδομένων και αυτόματη εναλλαγή κλειδιού (Key Rotation).

5.2 Έλεγχος Παραβίασης Ακεραιότητας (Tamper Detection)

Ένα από τα σημαντικότερα χαρακτηριστικά της εφαρμογής είναι η ικανότητα ανίχνευσης αλλοίωσης των δεδομένων (*Integrity Protection*). Για τη δοκιμή αυτή, πραγματοποιήθηκε χειροκίνητη τροποποίηση (*bit flipping*) ενός byte στο κρυπτογραφημένο αρχείο `secret.txt.enc`, προκειμένου να προσομοιωθεί μια κακόβουλη αλλαγή στα δεδομένα.

Στο Σχήμα 5.3 παρουσιάζεται η αντίδραση του συστήματος σε αυτή την παραβίαση. Η εφαρμογή, μέσω του μηχανισμού AEAD (ή του HMAC-SHA256 ανάλογα με το Mode), εντοπίζει άμεσα ότι το *Authentication Tag* δεν αντιστοιχεί πλέον στο περιεχόμενο. Το σύστημα διακόπτει άμεσα τη διαδικασία προτού παραχθεί λανθασμένο *plaintext*, εκδίδοντας προειδοποίηση για αποτυχία αυθεντικοποίησης (*Authentication Failed*).

```
Enter selection (1-2): 2

Enter file path: secret.txt.enc
Password:
[!] Authentication Failed: Wrong password or corrupted metadata.
(venv) PS C:\Users\katsi\Documents\Projects\secure-file-rotation-python\src>
```

Εικόνα 5.3: Απόρριψη αποκρυπτογράφησης μετά από ανίχνευση αλλοίωσης στα δεδομένα του αρχείου (Tamper Detection).

5.3 Δοκιμές Key Rotation (Verification of Key Change)

Η τελευταία δοκιμή αφορά τον μηχανισμό ανανέωσης κλειδιών (*Key Rotation*), ο οποίος είναι κρίσιμος για τη διατήρηση της μακροπρόθεσμης ασφάλειας των αρχείων.

Όπως φαίνεται στο Σχήμα 5.4, κατά την εκτέλεση της λειτουργίας *Key Rotation*, το σύστημα ζητά τον παλαιό (*Old Password*) και τον νέο κωδικό πρόσβασης (*New Password*). Μετά την επιτυχή επαλήθευση του παλαιού κωδικού, το αρχείο επανακρυπτογραφείται με νέο *Salt* και νέο κλειδί. Στη συγκεκριμένη δοκιμή χρησιμοποιήθηκε ο αλγόριθμος `ChaCha20-Poly1305`, αποδεικνύοντας την ευελιξία (*Cipher Agility*) του συστήματος. Η επιτυχία της δοκιμής επιβεβαιώνεται από το γεγονός ότι ο παλαιός κωδικός δεν μπορεί πλέον να αποκρυπτογραφήσει το αρχείο, διασφαλίζοντας το «*Forward Secrecy*» των δεδομένων.

```
Enter selection (1-2): 2

Enter file path: secret.txt.enc
Old Password:
New Password:
[+] Mode 3 (ChaCha20-Poly1305) complete: secret.txt.enc (Metadata Integrity Secured)
[+] Key Rotation Successful! New unique salt and master key generated.
(venv) PS C:\Users\katsi\Documents\Projects\secure-file-rotation-python\src>
```

Εικόνα 5.4: Επιτυχής ανανέωση κρυπτογραφικού κλειδιού με χρήση του αλγορίθμου ChaCha20-Poly1305 και παραγωγή νέου Master Key.

5.4 Δυνατότητα Συνεχόμενης Επεξεργασίας (CLI Loop)

Προκειμένου να καλυφθεί η απαίτηση για την επεξεργασία πολλαπλών αρχείων, η διεπαφή γραμμής εντολών (*CLI*) υλοποιήθηκε με τη χρήση ενός βρόχου επανάληψης (`while loop`). Η προσέγγιση αυτή επιτρέπει στον χρήστη να εκτελεί διαδοχικές κρυπτογραφικές λειτουργίες σε διαφορετικά αρχεία, χωρίς να απαιτείται η επανεκκίνηση της εφαρμογής.

Όπως φαίνεται στο Σχήμα 5.5, μετά την ολοκλήρωση μιας λειτουργίας, το πρόγραμμα επιστρέφει αυτόματα στο κεντρικό μενού, δίνοντας στον χρήστη την επιλογή να συνεχίσει με νέο αρχείο ή να τερματίσει την εκτέλεση επιλέγοντας «*Exit*». Η δυνατότητα αυτή καθιστά την εφαρμογή κατάλληλη για μαζικότερη χρήση (*bulk processing capability*) σε περιβάλλοντα παραγωγής.

```
[!] Security Level:
[1] 128-bit
[2] 256-bit

Enter selection (1-2): 2

Enter file path: secret.txt
Password:
[+] Mode 3 (ChaCha20-Poly1305) complete: secret.txt.enc (Metadata Integrity Secured)
[+] Original file deleted for security.

==========================================
          FILE PROTECTION TOOL
==========================================

[!] Select Mode:
 1. Confidentiality (AES-CTR)
 2. Integrity (HMAC-SHA256)
 3. Confidentiality & Integrity (Combined)
 4. Rotate Key / Change Password
 5. EXIT

Enter selection (1-5): 5

[+] Exiting... Goodbye!
(venv) PS C:\Users\katsi\Documents\Projects\secure-file-rotation-python\src>
```

Εικόνα 5.5: Εμφάνιση του ανανεωμένου μενού και δυνατότητα επιλογής επόμενου αρχείου ή τερματισμού.

6 Κεφάλαιο 6: Τεχνικές Προκλήσεις

Μετά την πρακτική επαλήθευση της ορθής λειτουργίας του συστήματος και την επιτυχή ολοκλήρωση των δοκιμών που παρουσιάστηκαν στο προηγούμενο κεφάλαιο, η ανάλυση προχωρά στην αξιολόγηση των τεχνικών προκλήσεων που αναδείχθηκαν κατά την ανάπτυξη. Στο παρόν κεφάλαιο εξετάζονται οι δυσκολίες που αντιμετωπίστηκαν στη διαχείριση των κρυπτογραφικών βιβλιοθηκών, ενώ πραγματοποιείται μια συγκριτική αποτίμηση της αποδοτικότητας μεταξύ των διαθέσιμων αλγορίθμων. Τέλος, καταγράφονται οι περιορισμοί του συστήματος, προσφέροντας μια ειλικρινή αποτίμηση της τρέχουσας υλοποίησης και θέτοντας τις βάσεις για τις μελλοντικές βελτιώσεις που θα καταστήσουν την εφαρμογή ακόμη πιο στιβαρή και αποδοτική.

6.1 Αντιμετώπιση Προβλημάτων (Troubleshooting)

Κατά τη διάρκεια της ανάπτυξης και των δοκιμών της εφαρμογής, αντιμετωπίστηκαν ορισμένες τεχνικές προκλήσεις που απαιτούσαν προσεκτικό χειρισμό:

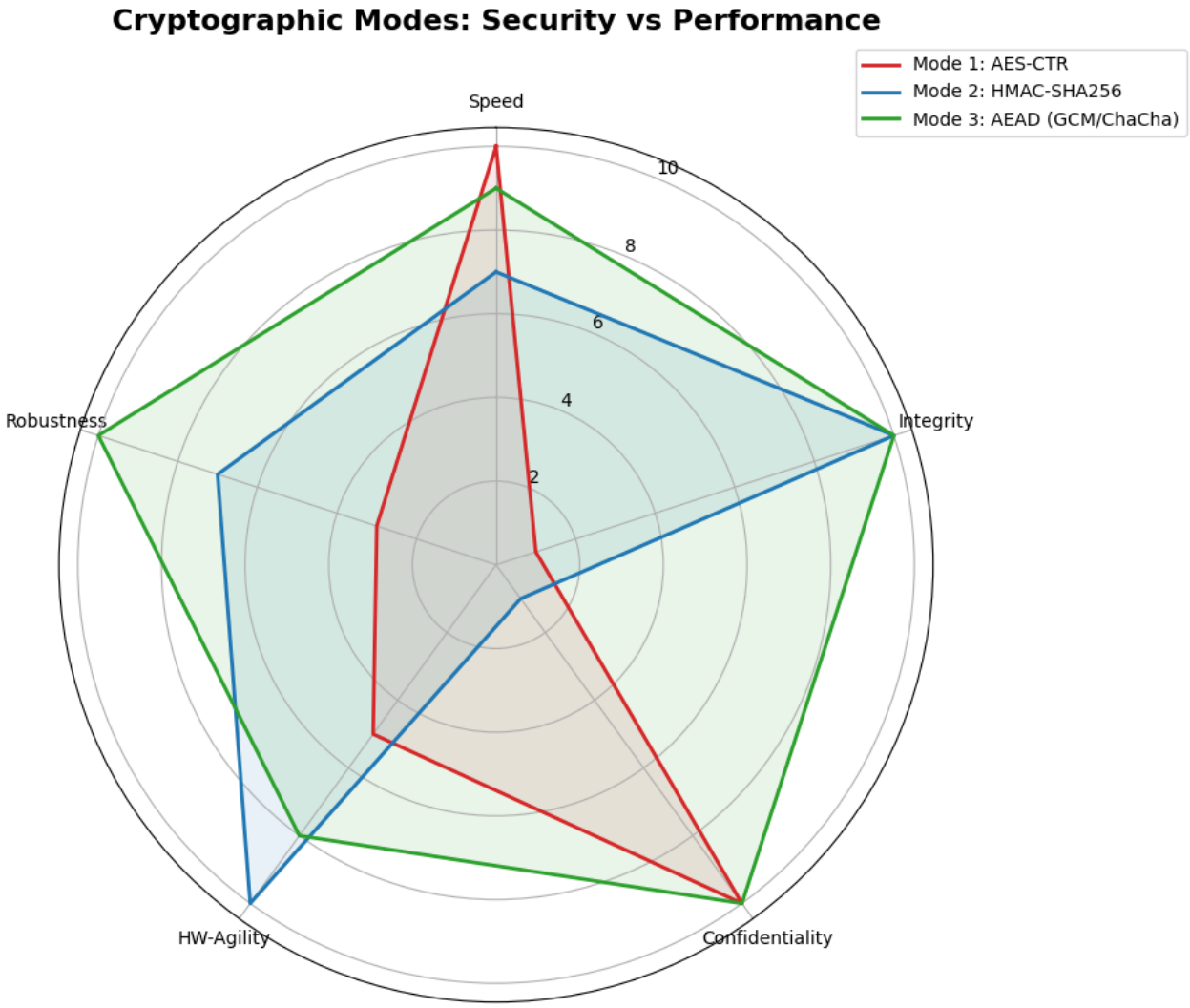
- Συμβατότητα Βιβλιοθηκών: Η εγκατάσταση της βιβλιοθήκης argon2-cffi σε περιβάλλον Windows απαιτούσε συγκεκριμένες ρυθμίσεις για να μπορέσει η Python να "συνομιλήσει" με τον αλγόριθμο Argon2id. Το πρόβλημα λύθηκε με τη χρήση ενός εικονικού περιβάλλοντος (Virtual Environment - venv), το οποίο εξασφάλισε ότι όλες οι εξαρτήσεις είναι απομονωμένες και σταθερές.
- Διαχείριση Δυαδικών Δεδομένων (Binary Data): Μια βασική δυσκολία ήταν η σωστή ανάγνωση και εγγραφή των Metadata (Salt, Nonce) μαζί με το κρυπτογραφημένο περιεχόμενο στο ίδιο αρχείο. Για να αποφευχθεί η καταστροφή των δεδομένων, επιλέχθηκε η αποκλειστική χρήση του Binary Mode (rb/wb), διασφαλίζοντας ότι καμία πληροφορία δεν θα αλλοιωθεί από λανθασμένη κωδικοποίηση χαρακτήρων.
- Ασφάλεια κατά το Key Rotation: Υπήρχε ο κίνδυνος να καταστραφεί ένα αρχείο αν η διαδικασία αλλαγής κλειδιού διακοπτόταν στη μέση. Για την αντιμετώπιση αυτού, η εφαρμογή υλοποιήθηκε έτσι ώστε να ολοκληρώνει πρώτα την αποκρυπτογράφηση στη μνήμη και, μόνο αφού επιβεβαιωθεί ότι ο κωδικός είναι σωστός, να προχωρά στην εγγραφή των νέων δεδομένων στον δίσκο.

6.2 Σύγκριση Αποδοτικότητας Αλγορίθμων

Στην ενότητα αυτή παρουσιάζεται η σύγκριση μεταξύ των δύο βασικών αλγορίθμων που χρησιμοποιήθηκαν (AES-GCM και ChaCha20-Poly1305), όπως προέκυψε από τις δοκιμές αποδοτικότητας. Στο Σχήμα 6.1, φαίνεται ότι κάθε αλγόριθμος έχει τα δικά του πλεονεκτήματα:

- AES-GCM: Αποδείχθηκε ταχύτερος σε περιβάλλοντα που υποστηρίζουν hardware επιτάχυνση, καθώς είναι ο "στάνταρ" αλγόριθμος για σύγχρονους επεξεργαστές.
- ChaCha20-Poly1305: Αν και ελαφρώς πιο αργός σε κάποια τεστ, παρουσίασε πολύ σταθερή απόδοση χωρίς να απαιτεί ειδικό hardware, κάτι που τον καθιστά πολύ αξιόπιστο για γενική χρήση.

Όπως προκύπτει από το Radar Chart της Συγκριτικής Ανάλυσης στην Εικόνα 6.1, η επιλογή του Mode 3 (AEAD) αποτελεί τη βέλτιστη λύση για την προστασία αρχείων. Ενώ το Mode 1 προσφέρει ταχύτητα και το Mode 2 ακεραιότητα, μόνο το Mode 3 επιτυγχάνει τον συνδυασμό των δύο (Authenticated Encryption), διασφαλίζοντας ότι οποιαδήποτε προσπάθεια αλλοίωσης των δεδομένων θα εντοπιστεί αμέσως κατά τη διαδικασία της αποκρυπτογράφησης.



Εικόνα 6.1: Συγκριτική ανάλυση κρυπτογραφικών προτύπων λειτουργίας (Modes of Operation) ως προς την ασφάλεια και την αποδοτικότητα.

6.3 Περιορισμοί του Συστήματος

Παρά την επιτυχή υλοποίηση των κρυπτογραφικών λειτουργιών, η παρούσα εφαρμογή διαθέτει ορισμένους περιορισμούς που θα μπορούσαν να αποτελέσουν αντικείμενο μελλοντικής βελτίωσης:

- Περιορισμός Μνήμης RAM: Λόγω του τρόπου που διαβάζονται τα δεδομένα (φόρτωση όλου του αρχείου στη μνήμη), η εφαρμογή είναι κατάλληλη για αρχεία μικρού και μεσαίου μεγέθους. Για εξαιρετικά μεγάλα αρχεία (π.χ. αρκετών Gigabytes), θα απαιτούνταν η υλοποίηση "Chunked Processing" (επεξεργασία σε μικρά κομμάτια).
- Διαχείριση Μονοπατιών (File Paths): Η εφαρμογή απαιτεί ο χρήστης να βρίσκεται στον ίδιο φάκελο με το αρχείο ή να παρέχει το ακριβές όνομα του αρχείου. Δεν διαθέτει γραφικό περιβάλλον (GUI) ή προηγμένο File Explorer για την επιλογή αρχείων από διαφορετικές τοποθεσίες του συστήματος.
- Ανάκτηση Κωδικού: Όπως σε κάθε αυστηρά κρυπτογραφικό σύστημα, δεν υπάρχει μηχανισμός ανάκτησης (Password Recovery) σχεδιαστικά (by design). Αυτό διασφαλίζει ότι κανένας τρίτος, συμπεριλαμβανομένου του δημιουργού του λογισμικού, δεν μπορεί να αποκτήσει πρόσβαση στα δεδομένα. Σε περίπτωση απώλειας του συνθηματικού, τα δεδομένα καθίστανται μόνιμα μη προσβάσιμα.

Συμπεράσματα και Μελλοντικές επεκτάσεις

Συμπεράσματα Η παρούσα εργασία κατέδειξε ότι η ασφάλεια των δεδομένων δεν βασίζεται μόνο στην ισχύ των κρυπτογραφικών αλγορίθμων, αλλά κυρίως στον σωστό συνδυασμό τους και στη στιβαρή διαχείριση των παραμέτρων τους. Μέσα από την ανάπτυξη της εφαρμογής, υλοποιήθηκε ένα σύστημα που προσφέρει Agility (εναλλαγή αλγορίθμων), Ακεραιότητα (μέσω AEAD και HMAC) και Δυναμική Ασφάλεια (μέσω Key Rotation). Η χρήση του Argon2id ως μηχανισμού παραγωγής κλειδιών διασφαλίζει ανθεκτικότητα έναντι επιθέσεων brute-force, ενώ ο χειρισμός των εξαιρέσεων εγγυάται ότι ο χρήστης προστατεύεται από αλλοιωμένα δεδομένα. Το τελικό αποτέλεσμα είναι ένα λειτουργικό και ασφαλές εργαλείο, ικανό να θωρακίσει ευαίσθητες πληροφορίες σε τοπικό επίπεδο.

Μελλοντικές Επεκτάσεις Παρά την πληρότητα της τρέχουσας έκδοσης, υπάρχουν σημαντικά περιθώρια αναβάθμισης που θα μπορούσαν να καταστήσουν την εφαρμογή πιο χρηστική και ανθεκτική:

- ***Directory-level Encryption:*** Η χρήση των βιβλιοθηκών `os` ή `pathlib` θα επέτρεπε το *recursive traversal* φακέλων, εφαρμόζοντας μαζική κρυπτογράφηση και *Key Rotation* σε ολόκληρα δέντρα καταλόγων, αυτοματοποιώντας τη διαδικασία προστασίας μεγάλου όγκου δεδομένων.
- ***Ανάπτυξη Γραφικού Περιβάλλοντος (GUI):*** Η προσθήκη ενός περιβάλλοντος διεπαφής (π.χ. μέσω των βιβλιοθηκών `PyQt` ή `Tkinter`) θα καθιστούσε την εφαρμογή προσιτή σε μη τεχνικούς χρήστες, ενσωματώνοντας λειτουργίες όπως το «drag-and-drop» αρχείων για άμεση κρυπτογράφηση.
- ***Υποστήριξη Επιπλέον Αλγορίθμων:*** Υποστήριξη Επιπλέον Αλγορίθμων: Η ενσωμάτωση αλγορίθμων όπως ο `AES-SIV` ή ο `Camellia`, ο οποίος αποτελεί το ευρωπαϊκό και ιαπωνικό αντίστοιχο του AES με παρόμοια επίπεδα ασφάλειας, θα μπορούσε να προσφέρει ακόμη μεγαλύτερη ευελιξία (Cipher Agility) στο σύστημα.
- ***Chunked Encryption:*** Η τροποποίηση της ροής δεδομένων ώστε η κρυπτογράφηση να εκτελείται σε τμήματα (*chunks*) θα επέτρεπε τη διαχείριση αρχείων μεγέθους πολλών Gigabytes, ξεπερνώντας τους τρέχοντες περιορισμούς της μνήμης RAM.

Παράρτημα Α

Πίνακας 2: Ανάλυση Κρυπτογραφικών Μηχανισμών και Στόχων Προστασίας

Ακρωνύμιο	Τι είναι & Πώς λειτουργεί	Στόχος	Πλεονέκτημα (Τρικ)
AES	Block Cipher που κλειδώνει δεδομένα σε σταθερά "κουτιά" των 16 bytes.	Κρυπτογράφηση	Παγκόσμιο βιομηχανικό πρότυπο (NIST).
CTR Mode	Μετατρέπει τον AES σε Stream Cipher χρησιμοποιώντας έναν μετρητή.	Εμπιστευτικότητα	No Padding - Υποστηρίζει Παραλληλοποίηση.
GMAC	Μαθηματική συνάρτηση Galois που παράγει ψηφιακή σφραγίδα 16 bytes.	Ακεραιότητα	Bit-flipping detection (άμεση εύρεση αλλοίωσης).
GCM (AEAD)	Ο συνδυασμός CTR και GMAC. Ο απόλυτος μηχανισμός AEAD.	Confidentiality & Integrity	Ταχύτητα και ασφάλεια σε ένα πέρασμα.
AES-NI	Ειδικά κυκλώματα (Hardware) μέσα στον επεξεργαστή για πράξεις AES.	Απόδοση	Μηδενικό latency ακόμα και με κλειδί 256 bits.

Πίνακας 3: Προδιαγραφές Συναρτήσεων Κρυπτογραφικού Μηχανισμού

Όνομα Συνάρτησης	Παράμετροι	Κρυπτογραφικός Σκοπός
`_derive_master_key`	passphrase, salt	**KDF Engine:** Χρήση Argon2id για τη μετατροπή κωδικού σε κλειδί 128/256-bit.
`protect_file`	path, pass, data, del	**Mode 3 (Protect):** Υλοποίηση AEAD. Ενσωμάτωση Salt και Nonce στα Associated Data (AD).
`unprotect_file`	file_path, passphrase	**Mode 3 (Unprotect):** Αποκρυπτογράφηση και έλεγχος ακεραιότητας. Key Rotation μετά την επιτυχία.
`sign_file`	file_path, pass, del	**Mode 2 (Integrity):** Υλοποίηση HMAC-SHA256 για δημιουργία ψηφιακής υπογραφής.
`verify_file`	file_path, passphrase	**Mode 2 (Verify):** Έλεγχος εγκυρότητας HMAC και αυτόματο re-signing (Key Rotation).
`protect_conf_only`	file_path, pass, del	**Mode 1 (Confidentiality):** AES-CTR κρυπτογράφηση (Εμπιστευτικότητα χωρίς Ακεραιότητα).
`rotate_key`	path, old_p, new_p	**Manual Rotation:** Αλλαγή Master Password με επανακρυπτογράφηση μέσω νέου salt.

Παράρτημα Β: Github repository

Για την υλοποίηση και τη διευκόλυνση της αναπαραγωγής των αποτελεσμάτων, το σύνολο των αρχείων ρυθμίσεων (configuration files), των scripts και του κώδικα Python, έχει αναρτηθεί στο προσωπικό μου repository στο GitHub.

Στο repository περιλαμβάνονται:

- **Πηγαίος Κώδικας** (`src/`): Η κλάση `SecureFileProtector` και το CLI interface για την αλληλεπίδραση με τον χρήστη.
- **Τεχνική Τεκμηρίωση** (`docs/`): Τα διαγράμματα αρχιτεκτονικής σε μορφή `.png` και το πρωτότυπο αρχείο σχεδίασης `.drawio`.
- **Επίδειξη Λειτουργίας (Video Demo)**: Ένα αρχείο βίντεο (`PoC_demo.mp4`) που παρουσιάζει ζωντανά τη διαδικασία κρυπτογράφησης, την ανίχνευση αλλοίωσης (tamper detection) και το αυτόματο key rotation.
- **Αρχείο Εξαρτήσεων** (`requirements.txt`): Λίστα με όλες τις απαραίτητες εξωτερικές βιβλιοθήκες (`cryptography`, `argon2-cffi`) για την ορθή εκτέλεση του προγράμματος.
- **Οδηγίες Χρήσης (README.md)**: Αναλυτικό εγχειρίδιο για την εγκατάσταση των εξαρτήσεων και την εκτέλεση των διαφόρων λειτουργιών της εφαρμογής.

Σύνδεσμος Αποθετηρίου:

https://github.com/drososkats/secure-file-rotation-python

Βιβλιογραφικές Αναφορές

[1] Harokopio University of Athens. *Course Materials: Distributed Systems (DIT292)*. https://eclass.hua.gr/modules/document/?course=DIT286. 2024.

[2] National Institute of Standards and Technology (NIST). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST Special Publication 800-38D. 2007. URL: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf.

[3] Y. Nir and A. Langley. *ChaCha20 and Poly1305 for IETF Protocols*. Internet Engineering Task Force (IETF). 2018. URL: https://datatracker.ietf.org/doc/html/rfc8439.

[4] Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich. *Argon2: the memory-hard function for password hashing and other applications*. GitHub / PHC winner resources. 2015. URL: https://github.com/P-H-C/phc-winner-argon2.

[5] Bruce Schneier. *Applied Cryptography: Protocols, Algorithms, and Source Code in C*. 20th Anniversary. Wiley, 2015.